

Supporting Document

Mandatory Technical Document

collaborative Protection Profile for Full Drive Encryption – Encryption Engine

Version: 3.0

2026-03-12

Full Disk Encryption international Technical Community

Foreword

This is a Supporting Document (SD), intended to complement the Common Criteria version 3 and the associated Common Evaluation Methodology for Information Technology Security Evaluation.

SDs may be “Guidance Documents”, that highlight specific approaches and application of the standard to areas where no mutual recognition of its application is required, and as such, are not of normative nature, or “Mandatory Technical Documents”, whose application is mandatory for evaluations whose scope is covered by that of the SD. The usage of the latter class is not only mandatory, but certificates issued as a result of their application are recognized under the CCRA.

Technical Editor:

National Information Assurance Partnership (NIAP)

Document history:

Version	Date	Comment
0.1	2014-08-26	Initial release for iTC review
0.2	2014-09-05	Draft published for public review
0.13	2014-10-17	Incorporated comments received from the public review
1.0	2015-01-26	Incorporated comments received from the CCDB review
1.5	2015-09-02	Revised based on additional use cases developed by iTC
2.0	2016-09-09	Incorporated comments received from the public review, and also updated the Key Destruction section and AVA_VAN.

2.0 + Errata 20190201	2019-02-01	Updated to reflect CC Part 3 evaluation findings and FDE Interpretation Team [FIT] rulings
3.0	2026-03-12	Updated for CC:2022, reviewing and applying modifications to the cPP

General Purpose:

The purpose of this SD is to define evaluation methods for the functional behavior of products.

Acknowledgments:

This SD was developed with support from NIAP Technical Community members, with representatives from industry, government agencies, Common Criteria Test Laboratories, and members of academia.

Table of Contents

- 1 Introduction
 - 1.1 Technology Area and Scope of Supporting Document
 - 1.2 Structure of the Document
 - 1.3 Terms
 - 1.3.1 Common Criteria Terms
 - 1.3.2 Technical Terms
- 2 Evaluation Activities for SFRs
 - 2.1 TOE SFR Evaluation Activities
 - 2.1.1 Cryptographic Support (FCS)
 - 2.1.2 User Data Protection
 - 2.1.3 Security Management (FMT)
 - 2.1.4 Protection of the TSF (FPT)
 - 2.2 Evaluation Activities for Optional SFRs
 - 2.2.1 Protection of the TSF (FPT)
 - 2.3 Evaluation Activities for Selection-Based SFRs
 - 2.3.1 Cryptographic Support (FCS)
 - 2.3.2 Protection of the TSF (FPT)
 - 2.4 Evaluation Activities for Objective SFRs
 - 2.5 Evaluation Activities for Implementation-dependent SFRs
 - 2.5.1 Cryptographic Support (FCS)
- 3 Evaluation Activities for SARs
 - 3.1 ADV: Development
 - 3.2 AGD: Guidance Documentation
 - 3.3 Class ALC: Life-cycle Support
 - 3.4 Class ATE: Tests
 - 3.5 Class AVA: Vulnerability Assessment
- 4 Required Supplementary Information
- Appendix A - References

1 Introduction

1.1 Technology Area and Scope of Supporting Document

The scope of the collaborative Protection Profile for Full Drive Encryption – Encryption Engine is to describe the security functionality of products in terms of [CC] and to define functional and assurance requirements for them.

Although Evaluation Activities are defined mainly for the evaluators to follow, in general they also help developers to prepare for evaluation by identifying specific requirements for their TOE. The specific requirements in Evaluation Activities may in some cases clarify the meaning of Security Functional Requirements (SFR), and may identify particular requirements for the content of Security Targets (ST) (especially the TOE Summary Specification), user guidance documentation, and possibly supplementary information (e.g. for entropy analysis or cryptographic key management architecture).

1.2 Structure of the Document

Evaluation Activities can be defined for both SFRs and Security Assurance Requirements (SAR), which are themselves defined in separate sections of the SD.

If any Evaluation Activity cannot be successfully completed in an evaluation, then the overall verdict for the evaluation is a 'fail'. In rare cases there may be acceptable reasons why an Evaluation Activity may be modified or deemed not applicable for a particular TOE, but this must be approved by the Certification Body for the evaluation.

In general, if all Evaluation Activities (for both SFRs and SARs) are successfully completed in an evaluation then it would be expected that the overall verdict for the evaluation is a 'pass'. To reach a 'fail' verdict when the Evaluation Activities have been successfully completed would require a specific justification from the evaluator as to why the Evaluation Activities were not sufficient for that TOE.

Similarly, at the more granular level of assurance components, if the Evaluation Activities for an assurance component and all of its related SFR Evaluation Activities are successfully completed in an evaluation then it would be expected that the verdict for the assurance component is a 'pass'. To reach a 'fail' verdict for the assurance component when these Evaluation Activities have been successfully completed would require a specific justification from the evaluator as to why the Evaluation Activities were not sufficient for that TOE.

1.3 Terms

The following sections list Common Criteria and technology terms used in this document.

1.3.1 Common Criteria Terms

Assurance	Grounds for confidence that a TOE meets the SFRs [CC] .
Base Protection Profile (Base-PP)	Protection Profile used as a basis to build a PP-Configuration.
Collaborative Protection Profile (cPP)	A Protection Profile developed by international technical communities and approved by multiple schemes.
Common Criteria (CC)	Common Criteria for Information Technology Security Evaluation (International Standard ISO/IEC 15408).
Common Criteria Testing Laboratory	Within the context of the Common Criteria Evaluation and Validation Scheme (CCEVS), an IT security evaluation facility accredited by the National Voluntary Laboratory Accreditation Program (NVLAP) and approved by the NIAP Validation Body to conduct Common Criteria-based evaluations.
Common Evaluation Methodology (CEM)	Common Evaluation Methodology for Information Technology Security Evaluation.

Direct Rationale	A type of Protection Profile, PP-Module, or Security Target in which the security problem definition (SPD) elements are mapped directly to the SFRs and possibly to the security objectives for the operational environment. There are no security objectives for the TOE.
Extended Package (EP)	A deprecated document form for collecting SFRs that implement a particular protocol, technology, or functionality. See Functional Packages.
Functional Package (FP)	A document that collects SFRs for a particular protocol, technology, or functionality.
Operational Environment (OE)	Hardware and software that are outside the TOE boundary that support the TOE functionality and security policy.
Protection Profile (PP)	An implementation-independent set of security requirements for a category of products.
Protection Profile Configuration (PP-Configuration)	A comprehensive set of security requirements for a product type that consists of at least one Base-PP and at least one PP-Module.
Protection Profile Module (PP-Module)	An implementation-independent statement of security needs for a TOE type complementary to one or more Base-PPs.
Security Assurance Requirement (SAR)	A requirement to assure the security of the TOE.
Security Functional Requirement (SFR)	A requirement for security enforcement by the TOE.
Security Target (ST)	A set of implementation-dependent security requirements for a specific product.
Target of Evaluation (TOE)	The product under evaluation.
TOE Security Functionality (TSF)	The security functionality of the product under evaluation.
TOE Summary Specification (TSS)	A description of how a TOE satisfies the SFRs in an ST.

1.3.2 Technical Terms

Authorization Factor	A value that a user knows, has, or is (e.g. password, token, etc.) submitted to the TOE to establish that the user is in the community authorized to use the drive. This value is used in the derivation or decryption of the BEV and eventual decryption of the DEK. Note that these values may or may not be used to establish the particular identity of the user.
Authorized User	A user who has a valid Authorization Factor or legitimate physical possession of the drive.
Border Encryption Value (BEV)	A value passed from the FDE Authorization Acquisition (AA) to the FDE Encryption Engine (EE) intended to link the key chains of the two components.

Data Encryption Key (DEK)	A key used to encrypt data-at-rest.
Full Drive Encryption (FDE)	Refers to partitions of logical blocks of user accessible data as managed by the host system. FDE products encrypt all data (with certain exceptions) on the partition of the storage device and permits access to the data only after successful authorization. FDE solutions may leave a portion of the storage device unencrypted, for such things as the Master Boot Record (MBR) or other AA/EE pre-authentication software, so long as it contains no protected data.
Intermediate Key	A key used in a point between the initial user authorization and the DEK.
Key Chaining	The method of using multiple layers of encryption keys to protect data. A top layer key encrypts a lower layer key which encrypts the data; this method can have any number of layers.
Key Encryption Key (KEK)	A key used to encrypt other keys, such as DEKs or storage that contains keys.
Key Material	Key material is commonly known as critical security parameter (CSP) data, and also includes authorization data, nonces, and metadata.
Key Release Key (Krk)	A key used to release another key from storage, it is not used for the direct derivation or decryption of another key.
Key Sanitization	A method of sanitizing encrypted data by securely overwriting the key that was encrypting the data.
Non-Volatile Memory	A type of computer memory that will retain information without power.
Operating System (OS)	Software which runs at the highest privilege level and can directly control hardware resources.
Powered-Off State	The device has been shut down.
Protected Data	This refers to all encrypted data on the storage device. Protected data may exclude a Master Boot Record or Pre-authentication area of the drive – areas that are sometimes necessarily unencrypted.
Root of Trust for Update (RTU)	A type of RoT that verifies the integrity and authenticity of an update payload before initiating the update process.
Submask	A submask is a bit string that can be generated and stored in a number of ways. The individual SFRs provide context where these values may be used and what functions they perform, such as the BEV, intermediate key, or derived authentication value.

2 Evaluation Activities for SFRs

The EAs presented in this section capture the actions the evaluator performs to address technology specific aspects covering specific SARs (e.g. ASE_TSS.1, ADV_FSP.1, AGD_OPE.1, and ATE_IND.1) – this is in addition to the CEM workunits that are performed in [Section 3 Evaluation Activities for SARs](#).

Regarding design descriptions (designated by the subsections labeled TSS, as well as any required supplementary material that may be treated as proprietary), the evaluator must ensure there is specific information that satisfies the EA. For findings regarding the TSS section, the evaluator's verdicts will be associated with the CEM workunit ASE_TSS.1-1. Evaluator verdicts associated with the supplementary evidence will also be associated with ASE_TSS.1-1, since the requirement to provide such evidence is specified in ASE in the PP.

For ensuring the guidance documentation provides sufficient information for the administrators/users as it pertains to SFRs, the evaluator's verdicts will be associated with CEM workunits ADV_FSP.1-7, AGD_OPE.1-4, and AGD_OPE.1-5.

Finally, the subsection labeled Tests is where the authors have determined that testing of the product in the context of the associated SFR is necessary. While the evaluator is expected to develop tests, there may be instances where it is more practical for the developer to construct tests, or where the developer may have existing tests. Therefore, it is acceptable for the evaluator to witness developer-generated tests in lieu of executing the tests. In this case, the evaluator must ensure the developer's tests are executing both in the manner declared by the developer and as mandated by the EA. The CEM workunits that are associated with the EAs specified in this section are: ATE_IND.1-3, ATE_IND.1-4, ATE_IND.1-5, ATE_IND.1-6, and ATE_IND.1-7.

2.1 TOE SFR Evaluation Activities

2.1.1 Cryptographic Support (FCS)

FCS_CKM.1/DEK Cryptographic Key Generation (Data Encryption Key)

FCS_CKM.1/DEK

TSS

The evaluator shall examine the TSS to determine that it describes how the TOE obtains a DEK (either generating the DEK or receiving from the environment).

If the TOE generates a DEK, the evaluator shall review the TSS to determine that it describes how the functionality described by FCS_RBG.1 is invoked. If the DEK is generated outside of the TOE, the evaluator checks to ensure that for each platform identified in the TOE the TSS, it describes the interface used by the TOE to invoke this functionality. The evaluator uses the description of the interface between the RBG and the TOE to determine that it requests a key greater than or equal to the required key sizes.

If the TOE received the DEK from outside the OE, then the evaluator shall examine the TSS to determine that the DEK is sent wrapped using the appropriate encryption algorithm.

Guidance

There is no AGD for this activity.

KMD

If the TOE received the DEK from outside the OE, then the evaluator shall verify that the KMD describes how the TOE unwraps the DEK.

Tests

- Test FCS_CKM.1/DEK:1: The evaluator shall configure the TOE to ensure the functionality of all selections.

FCS_CKM.6/Power Cryptographic Key and Key Material Destruction (Power Management)

FCS_CKM.6.1/Power

TSS

The evaluator shall verify the TSS provides a description of what keys and key material are destroyed when entering any compliant power saving state.

Guidance

The evaluator shall validate that guidance documentation contains clear warnings and information on conditions in which the TOE may end up in a non-compliant power saving state indistinguishable from a compliant power saving state. In that case it must contain mitigation instructions on what to do in such scenarios.

KMD

The evaluator shall verify the KMD includes a description of the areas where keys and key material reside.

The evaluator shall verify the KMD includes a key lifecycle that includes a description where key material resides, how the key material is used, and how the material is destroyed once it is not needed and that the documentation in the KMD follows FCS_CKM_EXT.6 for the destruction.

Tests

There are no test evaluation activities for this SFR.

FCS_CKM.6.2/Power

TSS

The evaluator shall verify the TSS provides a high level description of how keys stored in volatile memory are destroyed. The evaluator to verify that TSS outlines:

- if and when the TSF or the Operational Environment is used to destroy keys from volatile memory;
- if and how memory locations for (temporary) keys are tracked;
- details of the interface used for key erasure when relying on the OE for memory erasure.

Guidance

The evaluator shall check the guidance documentation if the TOE depends on the Operational Environment for memory erasure and how that is achieved.

KMD

The evaluator shall check to ensure the KMD lists each type of key, its origin, possible memory locations in volatile memory.

Tests

There are no test evaluation activities for this SFR.

FCS_CKM_EXT.6 Cryptographic Key Destruction Types

FCS_CKM_EXT.6

TSS

(Key Management Description may be used if necessary details describe proprietary information)

The evaluator shall examine the TOE's keychain in the TSS/KMD and verify all keys subject to destruction are destroyed according to one of the specified methods.

Guidance

There is no AGD for this activity.

Tests

There is no test for this activity.

FCS_COP.1/Hash Cryptographic Operation (Hash Algorithm)

FCS_COP.1/Hash

TSS

The evaluator shall examine the TSS to verify that if SHA-256 is selected, that it is being used only as a PRF, including as part of a key derivation function or RBG, or as part of LMS or XMSS, and not as a hash algorithm.

Guidance

There are no AGD evaluation activities for this SFR.

KMD

There are no KMD evaluation activities for this SFR.

Tests

The following tests may require the developer to provide access to a test platform that provides the evaluator with tools that are typically not found on factory products.

The following tests are conditional based upon the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

SHA-256, SHA-384, SHA-512

To test the TOE's ability to generate hash digests using SHA2 the evaluator shall perform the Algorithm Functional Test, Monte Carlo Test, and Large Data Test for each claimed SHA2 algorithm.

Algorithm Functional Test

The evaluator shall generate a number of test cases equal to the block size of the hash (512 for SHA2-256; 1024 for the other SHA2 algorithms).

Each test case is to consist of random data of a random length between 0 and 65536 bits, or the largest size supported.

Each test case is to consist of random data of a random length between 0 and 65536 bits, or the largest size supported.

Monte Carlo Test

Monte Carlo tests begin with a single seed and run 100 iterations of the chained computation.

There are two versions of the Monte Carlo test for SHA-1 and SHA-2. Either one is acceptable. For the Standard Monte Carlo test the message hashed is always three times the length of the initial seed.

```
For j = 0 to 99
  A = B = C = SEED
  For i = 0 to 999
    MSG = A || B || C
    MD = SHA(MSG)
    A = B
    B = C
    C = MD
  Output MD
  SEED = MD
```

For the alternate version of the Monte Carlo Test, the hashed message is always the same length as the seed.

```
INITIAL_SEED_LENGTH = LEN(SEED)
For j = 0 to 99
  A = B = C = SEED
  For i = 0 to 999
    MSG = A || B || C
    if LEN(MSG) >= INITIAL_SEED_LENGTH:
      MSG = leftmost INITIAL_SEED_LENGTH bits of MSG
    else:
```



```

        MSG = MSG || INITIAL_SEED_LENGTH - LEN(MSG) 0 bits
    MD = SHA(MSG)
    A = B
    B = C
    C = MD
Output MD
SEED = MD

```

The evaluator shall compare the output against results generated by a known-good implementation with the same input.

Large Data Test

The implementation must be tested against one test case each on large data messages of 1GB, 2GB, 4GB, and 8GB of data as supported. The data need not be random. It may, for example, consist of a repeated pattern of 64 bits.

The evaluator shall compare the output against results generated by a known-good implementation with the same input.

SHA3-384, SHA3-512 To test the TOE's ability to generate hash digests using SHA3 the evaluator shall perform the Algorithm Functional Test, Monte Carlo Test, and Large Data Tests for each claimed SHA3 algorithm.

Algorithm Functional Test

Generate a test case consisting of random data for every message length from 0 bits (or the smallest supported message size) to rate bits, where rate equals

- 832 for SHA3-384 and
- 576 for SHA3-512.

Additionally, generate tests cases of random data for messages of every multiple of (rate+1) bits starting at length rate, and continuing until 65535 is exceeded.

The evaluator shall compare the output against results generated by a known-good implementation with the same input.

Monte Carlo Test

Monte Carlo tests begin with a single seed and run 100 iterations of the chained computation.

For this Monte Carlo Test, the hashed message is always the same length as the seed.

```

MD[0] = SEED
INITIAL_SEED_LENGTH = LEN(SEED)
For 100 iterations
    For i = 1 to 1000
        MSG = MD[i-1];
        if LEN(MSG) >= INITIAL_SEED_LENGTH:
            MSG = leftmost INITIAL_SEED_LENGTH bits of MSG
        else:
            MSG = MSG || INITIAL_SEED_LENGTH - LEN(MSG) 0 bits
        MD[i] = SHA3(MSG)
    MD[0] = MD[1000]
Output MD[0]

```

The evaluator shall compare the output against results generated by a known-good implementation with the same input.

Large Data Test

The implementation must be tested against one test case each on large data messages of 1GB, 2GB, 4GB, and 8GB of data as supported. The data need not be random. It may, for example, consist of a repeated pattern of 64 bits.

The evaluator shall compare the output against results generated by a known-good implementation with the same input.

FCS_COP.1/SigVer Cryptographic Operation - Signature Verification

FCS_COP.1/SigVer

TSS

The evaluator shall examine the TSS to verify that any one-time values such as nonces or masks are constructed and used in accordance with the relevant standards.

Guidance

There are no guidance activities for this SFR.

KMD

There are no KMD evaluation activities for this SFR.

Tests

The following tests are conditional based upon the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

RSA-PKCS Signature Verification

Identifier	Cryptographic Algorithm Parameters	Cryptographic Key Sizes	List of Standards
RSA-PKCS	RSASSA-PKCS1-v1_5	Modulus of size [selection: 3072, 4096, 6144, 8192] bits, hash [selection: SHA-384, SHA-512]	RFC 8017 (Section 8.2) [PKCS #1 v2.2] NIST FIPS PUB 186-5 (Section 5.4) [RSASSA-PKCS1-v1_5]

To test the TOE's ability to perform RSA Digital Signature Verification using PKCS1-v1_5 signature type, the evaluator shall perform Generated Data Test using the following input parameters:

- Modulus size [3072, 4096, 6144, 8192] bits
- Hash algorithm [SHA-384, SHA-512]

Generated Data Test

For each supported combination of the above parameters, the evaluator shall cause the TOE to generate six test cases using a random message and its signature such that the test cases are modified as follows:

- One test case is left unmodified
- For one test case the Message is modified
- For one test case the Signature is modified
- For one test case the exponent (e) is modified
- For one test case the IR is moved
- For one test case the Trailer is moved

The TOE must correctly verify the unmodified signatures and fail to verify the modified signatures.

RSA-PSS Signature Verification

Identifier	Cryptographic Algorithm Parameters	Cryptographic Key Sizes	List of Standards
RSA-PSS	RSASSA-PSS	Modulus of size [selection: 3072, 4096, 6144, 8192] bits, hash [selection: SHA-384, SHA-512]	RFC 8017 (Section 8.2) [PKCS #1 v2.2] NIST FIPS PUB 186-5 (Section 5.4) [RSASSA-PSS]

To test the TOE's ability to perform RSA Digital Signature Verification using PSS signature type, the evaluator shall perform the Generated Data Test using the following input parameters:

- Modulus size [3072, 4096, 6144, 8192] bits
- Hash algorithm [SHA-384, SHA-512]
- Salt length [0-hash length]
- Mask function [MGF1]

Generated Data Test

For each supported combination of the above parameters, the evaluator shall cause the TOE to generate six test cases using random data such that the test cases are modified as follows:

- One test case is left unmodified
- For one test case the Message is modified
- For one test case the Signature is modified
- For one test case the exponent (e) is modified
- For one test case the IR is moved
- For one test case the Trailer is moved

The TOE must correctly verify the unmodified signatures and fail to verify the modified signatures.

DSA Signature Verification

Identifier	Cryptographic Algorithm Parameters	Cryptographic Key Sizes	List of Standards
DSA	DSA	Domain parameters for (L, N) = [(3072, 256)] bits	FIPS PUB 186-4 (Section 4.7) [DSA Signature Verification]

To test the TOE's ability to perform DSA Digital Signature Verification, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

- (L, N) = (3072, 256)
- Hash algorithm [SHA-384, SHA-512]

Algorithm Functional Test

For each supported combination of the above parameters, the evaluator shall cause the TOE to generate 15 test cases consisting of messages and signatures such that the 15 test cases are modified as follows:

- Three test cases are left unmodified
- For three test cases the Message is modified
- For three test cases the key is modified
- For three test cases the r value is modified
- For three test cases the s value is modified

The TOE must correctly verify the unmodified signatures and fail to verify the modified signatures.

ECDSA Signature Verification

Identifier	Cryptographic Algorithm Parameters	Cryptographic Key Sizes	List of Standards
ECDSA	ECDSA	Elliptic Curve [selection: P-384, P-521] and hash function using [selection: SHA-384, SHA-512]	[selection: ISO/IEC 14888-3:2018 (Subclause 6.6), NIST FIPS PUB 186-5 (Sections 6.3.1, 6.4.1) [ECDSA] NIST SP-800 186 (Section 4) [NIST Curves]

To test the TOE's ability to perform ECDSA Digital Signature Verification, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

- Elliptic Curve [P-384, P-521]
- Hash algorithm [SHA-384, SHA-512]

Algorithm Functional Test

For each supported combination of the above parameters, the evaluator shall cause the TOE to generate test cases consisting of messages and signatures such that the 21 test cases are modified as follows:

- Three test cases are left unmodified
- For three test cases the Message is modified
- For three test cases the key is modified
- For three test cases the r value is modified
- For three test cases the s value is modified
- For three test cases the value r is zeroed
- For three test cases the value s is zeroed

The TOE must correctly verify the unmodified signatures and fail to verify the modified signatures.

LMS Signature Verification

Identifier	Cryptographic Algorithm Parameters	Cryptographic Key Sizes	List of Standards
LMS	LMS	Private key size = [selection: 192 bits with [selection: SHA256/192, SHAKE256/192], 256 bits with [selection: SHA-256, SHAKE256]], Winternitz parameter = [selection: 1, 2, 4, 8], and tree height = [selection: 5, 10, 15, 20, 25]	RFC 8554 [LMS] NIST SP 800-208 [parameters]

To test the TOE's ability to verify cryptographic digital signature using LMS, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

- Hash algorithm [SHA-256/192, SHAKE256/192, SHA-256, SHAKE256]
- Winternitz [1, 2, 4, 8]
- Tree height [5, 10, 15, 20, 25]

Algorithm Functional Test

For each supported combination of the above parameters, the evaluator shall generate 4 test cases consisting of signed messages and keys, such that

- One test case is unmodified (i.e. correct)
- For one test case modify the message, i.e. the message is different
- For one test case modify the signature, i.e. signature is different
- For one test case modify the signature header so that it is a valid header for a different LMS parameter set.

The TOE must correctly verify the unmodified test case and fail to verify the modified test cases.

XMSS Signature Verification

Identifier	Cryptographic Algorithm Parameters	Cryptographic Key Sizes	List of Standards
XMSS	XMSS	Private key size = [selection: 192 bits with [selection: SHA256/192, SHAKE256/192], 256 bits with [selection: SHA-256, SHAKE256]], and tree height = [selection: 10, 16, 20]	RFC 8391 [XMSS] NIST SP 800-208 [parameters]

To test the TOE's ability to verify digital signatures using XMSS or XMSS MT, the evaluator shall perform the XMSS digital signature verification test using the following input parameters:

- Hash algorithm [SHA-256/192, SHAKE256/192, SHA-256, SHAKE256]
- Tree height [10, 16, 20]

XMSS Digital Signature Verification Test

For each supported combination of the above parameters, the evaluator shall generate four test cases consisting of signed messages and keys, such that

- One test case is unmodified (i.e. correct)
- For one test case modify the message, i.e. the message is different
- For one test case modify the signature, i.e. signature is different
- For one test case modify the signature header so that it is a valid header for a different XMSS parameter set

The evaluator shall verify the correctness of the implementation by verifying that the TOE correctly verifies the unmodified test case and fails to verify the modified test cases.

ML-DSA Signature Verification

Identifier	Cryptographic Algorithm Parameters	Cryptographic Key Sizes	List of Standards
ML-DSA	ML-DSA SigVer	Parameter set = ML-DSA-87	NIST FIPS PUB 204 (Section 5.2)

To test the TOE's ability to validate digital signatures using ML-DSA, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

- Parameter set [ML-DSA-87]
- Previously generated signed Message [8-65535] bytes
- *Mu* value (if generated externally)
- Context (for external interface testing)
- Previously generated Public key (*pk*)
- Previously generated Signature

Algorithm Functional Test

For each combination of supported parameter set and capabilities, the evaluator shall require the implementation under test to validate 15 signatures. Each group of 15 test cases is modified as follows:

- Three test cases are left unmodified
- For three test cases the Signed message is modified
- For three test cases the component of the signature that commits the signer to the message is modified
- For three test cases the component of the signature that allows the verifier to construct the vector *z* is modified
- For three test cases the component of the signature that allows the verifier to construct the hint array is modified

The TOE must correctly verify the unmodified signatures and fail to verify the modified signatures.

FCS_COP.1/SKC Cryptographic Operation (AES Data Encryption/Decryption)

FCS_COP.1/SKC

TSS

The evaluator shall examine the TSS to ensure that it describes the construction of any IVs, tweak values, and counters in conformance with the relevant specifications.

If AES-XTS is claimed then the evaluator shall examine the TSS to verify that the TOE creates full-length keys by methods that ensure that the two key halves are different and independent. If a GCM mode algorithm is selected, then the evaluator shall examine the TOE summary specification to confirm that it describes how the IV is generated and that the same IV is never reused to encrypt different plaintext pairs under the same key. The evaluator shall also confirm that for each invocation of GCM, the length of the plaintext is at most $(2^{32})-2$ blocks.

Guidance

There is no AGD for this activity.

KMD

There is no KMD for this activity.

Tests

The following tests require the developer to provide access to a test platform that provides the evaluator with tools that are typically not found on factory products.

The following tests are conditional based upon the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

AES-CBC

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
AES-CBC	AES in CBC mode with non-repeating and unpredictable IVs	256 bits	[selection: ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES] [selection: ISO/IEC 10116:2017 (Clause 7), NIST SP 800-38A] [CBC]

To test the TOE's ability to encrypt/decrypt data using AES in CBC mode, the evaluator shall perform Algorithm Functional Tests and Monte Carlo Tests using the following input parameters:

- Key size [256] bits
- Direction [encryption, decryption]

Algorithm Functional Tests

Algorithm Functional Tests are designed to verify the correct operation of the logical components of the algorithm implementation under normal operation using different block sizes. For AES-CBC, there are two types of AFTs:

Known-Answer Tests

For each combination of direction and claimed key size, the TOE must be tested using the GFSBox, KeySbox, VarTxt, and VarKey test cases listed in Appendixes B through E of *The Advanced Encryption Standard Algorithm Validation Suite (AESAVS)*, NIST, 15 November 2002.

Multi-Block Message Tests

For each combination of direction and claimed key size, the TOE must be tested against 10 test cases consisting of a random IV, random key, and random plaintext/ciphertext. The plaintext/ciphertext starts with a length of 16 bytes and increases by 16 bytes for each test case until reaching 160 bytes.

Monte Carlo Tests

Monte Carlo tests are intended to test the implementation under strenuous conditions. The TOE must process the test cases according to the following algorithm once for each combination of direction and key size:

```
Key[0] = Key
IV[0] = IV
PT[0] = PT
for i = 0 to 99 {
    Output Key[i], IV[i], PT[0]
    for j = 0 to 999 {
        if (j == 0) {
            CT[j] = AES-CBC-Encrypt(Key[i], IV[i], PT[j])
            PT[j+1] = IV[i]
        } else {
            CT[j] = AES-CBC-Encrypt(Key[i], PT[j])
            PT[j+1] = CT[j-1]
        }
    }
    Output CT[j]f
    AES_KEY_SHUFFLE(Key, CT)
    IV[i+1] = CT[j]
    PT[0] = CT[j-1]
}
```

where

AES_KEY_SHUFFLE

is defined as:

```
If ( keylen = 128 )
    Key[i+1] = Key[i] xor MSB(CT[j], 128)
If ( keylen = 192 )
    Key[i+1] = Key[i] xor (LSB(CT[j-1], 64) || MSB(CT[j], 128))
If ( keylen = 256 )
    Key[i+1] = Key[i] xor (MSB(CT[j-1], 128) || MSB(CT[j], 128))
```

The above pseudocode is for encryption. For decryption, swap all instances of CT and PT.

The initial IV, key, and plaintext/ciphertext should be random.

The evaluator shall test the decrypt functionality using the same test as above, exchanging CT and PT, and replacing AES-CBC-Encrypt with AES-CBC-Decrypt.

AES-XTS

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
AES-XTS	AES in XTS mode with unique tweak values that are consecutive non-negative integers starting at an arbitrary non-negative integer	512 bits	[selection: ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES] [selection: IEEE Std. 1619-2018, NIST SP 800-38E] [XTS]

To test the TOE's ability to encrypt/decrypt data using AES in XTS mode, the evaluator shall perform the Single Data Unit Test and the Multiple Data Unit Test using the following input parameters:

- Direction [encryption, decryption]
- Key size [512] bits
- Tweak value format [128-bit hex string, data unit sequence number]

Single Data Unit Test

For each combination of claimed key size, direction, and supported tweak value format, the evaluator shall generate 50 test cases consisting of random payload data. The payload data size is determined randomly for each test case from supported values within the range [128-65536] bits. The payload size and data unit size must be equal.

Multiple Data Unit Test

For each combination of claimed key size, direction, and supported tweak value format, the evaluator shall generate 50 test cases consisting of random payload data. The payload data size is determined randomly for each test case from supported values within the range [128-65536] bits. Likewise, the data unit size is determined randomly for each test case from supported values within the range [128-65535] bits. The payload size and data unit size must not be equal.

The evaluator shall verify the correctness of the TSF's implementation by comparing values generated by the TSF with those generated by a known good implementation using the same input parameters.

AES-GCM

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
AES-GCM	AES in GCM mode with non-repeating IVs using [selection: deterministic, RBG-based] IV construction; the tag must be of length [selection: 96, 104, 112, 120, or 128] bits.	256 bits	[selection: ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES] [selection: ISO/IEC 19772:2020 (Clause 10), NIST SP 800-38D] [GCM]

To test the TOE's implementation of AES-GCM authenticated encryption functionality the evaluator shall perform the Encryption Algorithm Functional Tests and Decryption Algorithm Functional Tests as described below using the following input parameters:

- Key Size [256] bits
- Associated data size [0-65536] bits
- Payload size [0-65536] bits
- IV size [96] bits
- Tag size [96, 104, 112, 120, 128] bits

Encryption Algorithm Functional Tests

The evaluator shall generate 15 test cases using random data for each combination of the above parameters as follows:

- Each claimed key size,
- Each supported tag size,
- Four supported non-zero payload sizes, such that two are multiples of 128 bits and two are not multiples of 128 bits,
- Four supported non-zero associated data sizes, such that two are multiples of 128 bits and two are not multiples of 128 bits, and
- An associated data size of zero, if supported.

Note that the IV size is always 96 bits.

The evaluator shall compare the output from each test case against results generated by a known- good implementation with the same input parameters.

Decryption Algorithm Functional Tests

The evaluator shall test the authenticated decrypt functionality of AES-GCM by supplying 15 test cases for the supported combinations of the parameters as described above. For each parameter combination the evaluator shall introduce an error into either the Ciphertext or the Tag such that approximately half of the cases are correct and half the cases contain errors.

FCS_KYC_EXT.2 Key Chaining (Recipient)

FCS_KYC_EXT.2

TSS

There is no TSS for this activity

Guidance

There is no AGD for this activity.

KMD

The evaluator shall examine the KMD to ensure it describes a high level key hierarchy and details of the key chain. The description of the key chain shall be reviewed to ensure it maintains a chain of keys using key wrap or key derivation methods that meet FCS_CKM.5, FCS_COP.1/KeyWrap, FCS_COP.1/KeyEncap, and/or FCS_COP.1/KeyEnc.

The evaluator shall examine the KMD to ensure that it describes how the key chain process functions, such that it does not expose any material that might compromise any key in the chain. (e.g. using a key directly as a compare value against a TPM) This description must include a diagram illustrating the key hierarchy implemented and detail where all keys and keying material is stored or what it is derived from. The evaluator shall examine the key hierarchy to ensure that at no point the chain could be broken without a cryptographic exhaust or knowledge of the

BEV and the effective strength of the DEK is maintained throughout the Key Chain.

The evaluator shall verify the KMD includes a description of the strength of keys throughout the key chain.

Tests

There is no test for this activity.

FCS_SNI_EXT.1 Cryptographic Operation (Salt, Nonce, and Initialization Vector Generation)

FCS_SNI_EXT.1

TSS

If salts are used, the evaluator shall ensure the TSS describes how salts are generated. The evaluator shall confirm that the salt is generating using an RBG described in FCS_RBG.1 or by the Operational Environment. If external function is used for this purpose, the TSS should include the specific API that is called with inputs.

If IVs or nonces are used, the evaluator shall ensure the TSS describes how nonces are created uniquely and how IVs and tweaks are handled (based on the AES mode). The evaluator shall confirm that the nonces are unique and the IVs and tweaks meet the stated requirements.

Guidance

There is no AGD for this activity.

KMD

There is no KMD for this activity.

Tests

There is no test for this activity

FCS_VAL_EXT.1 Validation

FCS_VAL_EXT.1.3

TSS

The evaluator shall examine the TSS to determine which authorization factors support validation.

The evaluator shall examine the TSS to review a high-level description if multiple submasks are used within the TOE, how the submasks are validated (e.g., each submask validated before combining, once combined validation takes place).

Guidance

(conditional) If the validation functionality is configurable, the evaluator shall examine the operational guidance to ensure it describes how to configure the TOE to ensure the limits regarding validation attempts can be established.

(conditional) If the validation functionality is specified by the ST author, the evaluator shall examine the operational guidance to ensure that it states the values that the TOE uses for limits regarding validation attempts.

KMD

The evaluator shall examine the KMD to verify that it described the method the TOE employs to limit the number of consecutively failed authorization attempts.

The evaluator shall examine the vendor's KMD to ensure it describes how validation is performed. The description of the validation process in the KMD provides detailed information how the TOE validates the submasks. The KMD describes how the process works, such that it does not expose any material that might compromise the submasks.

Tests

The evaluator shall perform the following tests:

- Test FCS_VAL_EXT.1.3:1: The evaluator shall determine the limit on the average rate of the number of consecutive failed authorization attempts. The evaluator will test the TOE by entering that number of incorrect authorization factors in consecutive attempts to access the protected data. If the limit mechanism includes any “lockout” period, the time period tested should include at least one such period. Then the evaluator will verify that the TOE behaves as described in the TSS.
- Test FCS_VAL_EXT.1.3:2: For each validated authorization factor, ensure that when the user provides an incorrect authorization factor, the TOE prevents the BEV from being forwarded outside the TOE (e.g., to the EE).

2.1.2 User Data Protection

FDP_DSK_EXT.1 Protection of Data on Disk

FDP_DSK_EXT.1

TSS

The evaluator shall examine the TSS to ensure that the description is comprehensive in how the data is written to the disk and the point at which the encryption function is applied. The TSS must make the case that standard methods of accessing the disk drive via the TOEs operating system will pass through these functions.

For the cryptographic functions that are provided by the Operational Environment, the evaluator shall check the TSS to ensure it describes, for each platform identified in the ST, the interfaces used by the TOE to invoke this functionality.

The evaluator shall verify the TSS in performing the evaluation activities for this requirement. The evaluator shall ensure the comprehensiveness of the description, confirms how the TOE writes the data to the disk drive, and the point at which it applies the encryption function.

The evaluator shall verify that the TSS describes the initialization of the TOE and the activities the TOE performs to ensure that it encrypts all the storage devices entirely when a user or administrator first provisions the TOE. The evaluator shall verify the TSS describes areas of the disk that it does not encrypt (e.g., portions associated with the Master Boot Records (MBRs), boot loaders, partition tables, etc.). If the TOE supports multiple disk encryptions, the evaluator shall examine the administration guidance to ensure the initialization procedure encrypts all storage devices on the platform.

Guidance

The evaluator shall review the AGD guidance to determine that it describes the initial steps needed to enable the FDE function, including any necessary preparatory steps. The guidance shall provide instructions that are sufficient, on all platforms, to ensure that all hard drive devices will be encrypted when encryption is enabled.

KMD

The evaluator shall verify the KMD includes a description of the data encryption engine, its components, and details about its implementation (e.g. for hardware: integrated within the device’s main SOC or separate co-processor, for software: initialization of the product, drivers, libraries (if applicable), logical interfaces for encryption/decryption, and areas which are not encrypted (e.g. boot loaders, portions associated with the Master Boot Record (MBRs), partition tables, etc.)). The evaluator shall verify the KMD provides a functional (block) diagram showing the main components (such as memories and processors) and the data path between, for hardware, the device’s host interface and the device’s persistent media storing the data, or for software, the initial steps needed to the activities the TOE performs to ensure it encrypts the storage device entirely when a user or administrator first provisions the product. The hardware encryption diagram shall show the location of the data encryption engine within the data path. The evaluator shall validate that the hardware encryption diagram contains enough detail showing the main components within the data path and that it clearly identifies the data encryption engine.

The evaluator shall verify the KMD provides sufficient instructions for all platforms to ensure that when the user enables encryption, the product encrypts all hard storage devices. The evaluator shall verify that the KMD describes the data flow from the device's host interface to the device's persistent media storing the data. The evaluator shall verify that the KMD provides information on those conditions in which the data bypasses the data encryption engine (e.g., read-write operations to an unencrypted Master Boot Record area).

The evaluator shall verify that the KMD provides a description of the platform's boot initialization, the encryption initialization process, and at what moment the product enables the encryption. The evaluator shall validate that the product does not allow for the transfer of user data before it fully initializes the encryption.

Tests

The evaluator shall perform the following tests:

- Test FDP_DSK_EXT.1:1:

The evaluator may require the software developer provides special tools which allow inspection of the encrypted drive either in-band or out-of-band.

- Step 1 Initialize the TOE: Ensure TOE is initialized and, if hardware, encryption engine is ready
 - Provision TOE to encrypt the storage device. Use a known key and the developer tools.
 - Determine a random character pattern of at least 64 KB
 - Retrieve information on what the device TOE's lowest and highest logical address is for which encryption is enabled.
- Step 2: Write pattern to storage device in multiple locations: Randomly select several logical address locations within the device's lowest to highest address range and write pattern to those addresses
- Step 3: Verify data is encrypted: Evaluator shall access storage using out-of-band tools bypassing encryption engine or inspecting the drive in a locked (secure encrypted) state to verify that data on storage media written to designated locations are not plaintext by comparing them with plaintext pattern.
- Step 4: Re-encrypt data: Evaluator shall enforce the TOE to re-encrypt data using new DEK by either:
 - changing the DEK and re-encrypting the data or
 - re-provisioning the device with new DEK and repeat step 2 by writing same data to same locations.
- Step 5: Verify data is encrypted using new DEK:
 - Evaluator shall access storage using out-of-band tools bypassing encryption engine or inspecting the drive in a locked (secure encrypted) state to verify that data on storage media written to designated locations are not plaintext by comparing them with plaintext pattern.
 - Evaluator shall verify that obtained ciphertext is different from the one obtained in Step 3 above. If same ciphertext is obtained, test is considered failing.

2.1.3 Security Management (FMT)

FMT_SMF.1 Specification of Management Functions

FMT_SMF.1

TSS

The evaluator shall ensure the TSS describes how the TOE changes the DEK.

The evaluator shall ensure the TSS describes how the TOE cryptographically erase the DEK.

The evaluator shall ensure the TSS describes the process to initiate TOE firmware/software updates.

If item d) is selected in FMT_SMF.1.1: If additional management functions are claimed in the ST, the evaluator shall verify that the TSS describes those functions.

Guidance

The evaluator shall review the AGD guidance and shall determine that the instructions for changing a DEK exist. The instructions must cover all environments on which the TOE is claiming conformance, and include any preconditions that must exist in order to successfully generate or re-generate the DEK.

The evaluator shall examine the operational guidance to ensure that it describes how to initiate TOE firmware/software updates.

If Disable Key Recovery is selected: The guidance for disabling key recovery shall be described in the AGD documentation.

KMD

If item d)) is selected in FMT_SMF.1.1: If the TOE offers the functionality to import an encrypted DEK, the evaluator shall ensure the KMD describes how the TOE imports a wrapped DEK and performs the decryption of the wrapped DEK.

Tests

The evaluator shall verify that the TOE has the functionality to change and cryptographically erase the DEK (effectively removing the ability to retrieve previous user data).

The evaluator shall verify that the TOE has the functionality to initiate TOE firmware/software updates.

If item d) is selected in FMT_SMF.1.1: If additional management functions are claimed, the evaluator shall verify that the additional features function as described.

2.1.4 Protection of the TSF (FPT)

FPT_KYP_EXT.1 Protection of Key and Key Material

FPT_KYP_EXT.1

TSS

The evaluator shall examine the TSS to verify that it describes the method by which intermediate keys are generated using submask combining.

Guidance

There are no AGD evaluation activities for this SFR.

KMD

The evaluator shall examine the KMD for a description of the methods used to protect keys stored in non-volatile memory.

The evaluator shall verify the KMD to ensure it describes the storage location of all keys and the protection of all keys stored in non-volatile memory. The description of the key chain shall be reviewed to ensure the selected method is followed for the storage of wrapped or encrypted keys in non-volatile memory and plaintext keys in non-volatile memory meet one of the criteria for storage.

Tests

If “only store keys in non-volatile memory when wrapped or encrypted” is selected, the evaluator shall verify that the TOE protects key material in a way consistent with the selections and description in the TSS and KMD. This can be achieved by inspecting non-volatile locations where the TOE stores key material or by inspecting the TOE memory when loading those materials from non-volatile storage. It would be sufficient to demonstrate that the stored value is different from the plaintext value of the key as discovered in the test EA for FCS_CKM.6.

FPT_PWR_EXT.1 Power Saving States

FPT_PWR_EXT.1

TSS

The evaluator shall validate the TSS contains a list of compliant power saving states.

Guidance

The evaluator shall ensure that guidance documentation contains a list of compliant power saving states. If additional power saving states are supported, then the evaluator shall validate that the guidance documentation states how the use of non-compliant power saving states can be avoided.

KMD

There is no KMD for this activity.

Tests

The evaluator shall confirm that for each listed compliant state all key/key materials are removed from volatile memory by using the test indicated by the selection in FCS_CKM_EXT.6.

FPT_PWR_EXT.2 Timing of Power Saving States

FPT_PWR_EXT.2

TSS

The evaluator shall validate that the TSS contains a list of conditions under which the TOE enters a compliant power saving state.

Guidance

The evaluator shall check that the guidance contains a list of conditions under which the TOE enters a Compliant power saving state. Additionally, the evaluator shall verify that the guidance documentation provides information on how long it is expected to take for the TOE to fully transition into the Compliant power saving state (e.g. how many seconds for the volatile memory to be completely erased).

KMD

There is no KMD for this activity.

Tests

The evaluator shall trigger each condition in the list of identified conditions and ensure the TOE ends up in a compliant power saving state by using the test indicated by the selection in FCS_CKM_EXT.6.

FPT_TST.1 TSF Self-Testing

FPT_TST.1

TSS

The evaluator shall examine the TSS to ensure that it details the self-tests that are run by the TSF along with how they are run. This description should include an outline of what the tests are actually doing. The evaluator shall ensure that the TSS makes an argument that the tests are sufficient to demonstrate that the DRBG is operating correctly.

Note that this information may also be placed in the entropy documentation specified by Appendix D - Entropy Documentation and Assessment.

Guidance

If a self-test can be executed at the request of an authorized user, the evaluator shall verify that the operational guidance provides instructions on how to execute that self-test.

Tests

For each self-test, the evaluator shall verify that evidence is produced that the self-test is executed when specified by FPT_TST.1.1.

If a self-test can be executed at the request of an authorized user, the evaluator shall verify that following the steps documented in the operational guidance to perform the self-test will result in execution of the self-test.

FPT_TUD_EXT.1 Trusted Update

FPT_TUD_EXT.1

TSS

If "authenticated firmware update mechanism as described in FPT_FUA_EXT.1" is selected, then FPT_FUA_EXT.1 must be claimed.

The evaluator shall examine the TSS to ensure that it describes information stating that an authorized source signs TOE updates and will have an associated digital signature. The evaluator shall examine the TSS contains a definition of an authorized source along with a description of how the TOE uses public keys for the update verification mechanism in the Operational Environment. The evaluator ensures the TSS contains details on the protection and maintenance of the TOE update credentials.

If the Operational Environment performs the signature verification, then the evaluator shall examine the TSS to ensure it describes, for each platform identified in the ST, the interfaces used by the TOE to invoke this cryptographic functionality.

Guidance

The evaluator ensures that the operational guidance describes how the TOE obtains vendor updates to the TOE; the processing associated with verifying the digital signature of the updates (as defined in FCS_COP.1/SigVer); and the actions that take place for successful and unsuccessful cases.

KMD

There are no KMD evaluation activities for this SFR.

Tests

The evaluators shall perform the following tests (if the TOE supports multiple signatures, each using a different hash algorithm, then the evaluator performs tests for different combinations of authentic and unauthentic digital signatures and hashes, as well as for digital signature alone):

- Test FPT_TUD_EXT.1:1: The evaluator performs the version verification activity to determine the current version of the TOE.
- Test FPT_TUD_EXT.1:2: The evaluator obtains a legitimate update using procedures described in the operational guidance and verifies that an update successfully installs on the TOE. The evaluator performs the version verification activity again to verify that the version correctly corresponds to that of the update.

2.2 Evaluation Activities for Optional SFRs

2.2.1 Protection of the TSF (FPT)

FPT_FAC_EXT.1 Firmware / Software Access Control

FPT_FAC_EXT.1

TSS

The evaluator shall examine the TSS to ensure that it describes information stating how the access control process takes place along with a description of the values or actions that are used.

Guidance

The evaluator ensures that the Operational Guidance describes how the user will be expected to interact with the authorization process.

KMD

There is no KMD for this activity.

Tests

The evaluator shall perform the following test.

- Test FPT_FAC_EXT.1:1: The evaluator shall try installing a upgrade and verify that a prompt is required and the appropriate value or action is necessary for the update to continue.

FPT_RBP_EXT.1 Rollback Protection

FPT_RBP_EXT.1

TSS

The evaluator shall examine the TSS to ensure that it describes at a high level the process for verifying that security version checking is performed before an upgrade is installed. The evaluator shall verify that a high level description of the types of error codes are provided and when an error would be triggered.

Guidance

The evaluator ensures that a description is provided on how the user should interpret the error codes.

KMD

There is no KMD for this activity.

Tests

The evaluator shall perform the following test:

- Test FPT_RBP_EXT.1:1: The evaluator shall try installing a lower security version number upgrade (either by just modifying the version number or by using an upgrade provided by the vendor) and will verify that the lower version cannot be installed and an error is presented to the user.

2.3 Evaluation Activities for Selection-Based SFRs

2.3.1 Cryptographic Support (FCS)

FCS_CKM.1/AKG Cryptographic Key Generation (Asymmetric Keys)

FCS_CKM.1/AKG

TSS

The evaluator shall ensure that the TSS identifies the key sizes / parameter sets supported by the TOE. If the ST specifies more than one scheme, the evaluator shall examine the TSS to verify that it identifies the usage for each scheme.

Guidance

The evaluator shall verify that the AGD guidance instructs the administrator how to configure the TOE to use the selected key generation schemes and key sizes for all uses specified by the AGD documentation and defined in this cPP.

KMD

If the TOE uses an asymmetric key as part of the key chain, the KMD should detail how the asymmetric key is used as part of the key chain.

Tests

If the ST selects "**implement functionality**," then the following test activities shall be carried out.

Evaluation Activity Note: The following tests may require the developer to provide access to a developer environment that provides the evaluator with tools that are not typically available to end-users of the application

Key Generation for FIPS PUB 186-5 RSA Schemes

The evaluator shall verify the implementation of RSA Key Generation by the TOE using the Key Generation test. This test verifies the ability of the TSF to correctly produce values for the key components including the public verification exponent e , the private prime factors p and q , the public modulus n and the calculation of the private signature exponent d . Key Pair generation specifies 5 ways (or methods) to generate the primes p and q . These include:

- Random Primes:
 - Provable primes
 - Probable primes
- Primes with Conditions:
 - Primes p_1, p_2, q_1, q_2, p , and q shall all be provable primes
 - Primes p_1, p_2, q_1 , and q_2 shall be provable primes, and p and q shall be probable primes
 - Primes p_1, p_2, q_1, q_2, p , and q shall all be probable primes

To test the key generation method for the Random Provable primes method and for all the Primes with Conditions methods, the evaluator must seed the TSF key generation routine with sufficient data to deterministically generate the RSA key pair. This includes the random seed(s), the public exponent of the RSA key, and the desired key length. For each key length supported, the evaluator shall have the TSF generate 25 key pairs. The evaluator shall verify the correctness of the TSF's implementation by comparing values generated by the TSF with those generated from a known good implementation.

If possible, the Random Probable primes method should also be verified against a known good implementation as described above. Otherwise, the evaluator shall have the TSF generate 10 keys pairs for each supported key length n_{len} and verify:

- $n = p \cdot q$,
- p and q are probably prime according to Miller-Rabin tests,
- $\text{GCD}(p-1, e) = 1$,
- $\text{GCD}(q-1, e) = 1$,
- $2^{16} \leq e \leq 2^{256}$ and e is an odd integer,
- $|p-q| > 2^{n_{len}/2 - 100}$,
- $p \geq 2^{n_{len}/2 - 1/2}$,
- $q \geq 2^{n_{len}/2 - 1/2}$,
- $2^{(n_{len}/2)} < d < \text{LCM}(p-1, q-1)$,
- $e \cdot d = 1 \bmod \text{LCM}(p-1, q-1)$.

Key Generation for Elliptic Curve Cryptography (ECC)

FIPS 186-5 ECC Key Generation Test- For each supported NIST curve, i.e., P-384 and P-521, the evaluator shall require the implementation under test (IUT) to generate 10 private/public key pairs. The private key shall be generated using an approved random bit generator (RBG). To determine correctness, the evaluator shall submit the generated key pairs to the public key verification (PKV) function of a known good implementation.

FIPS 186-5 Public Key Verification (PKV) Test- For each supported NIST curve, i.e., P-384 and P-521, the evaluator shall generate 10 private/public key pairs using the key generation function of a known good implementation and modify five of the public key values so that they are incorrect, leaving five values unchanged (i.e., correct). The evaluator shall obtain in response a set of 10 PASS/FAIL values.

Key Generation for ML-DSA

The evaluator shall 10x input to the internal KeyGen function a 32-byte random seed. Verify the returned public-private key pair is correct using a known good implementation. Here internal KeyGen refers to the TOE's

implementation of the function ML-DSA.KeyGen_internal(-) as described in FIPS.204.

Key Generation for ML-KEM

The evaluator shall 10x input to the internal KeyGen function a pair of 32-byte random string. Verify the returned encapsulation and decapsulation key pair is correct using a known good implementation. Here internal KeyGen refers to the TOE's implementation of the function ML-KEM.KeyGen_internal(-,-) as described in FIPS.203.

FCS_CKM.6/GENHW Cryptographic Key Destruction (General Hardware)

FCS_CKM.6/GENHW

TSS

(Key Management Description may be used if necessary details describe proprietary information)

The evaluator examines the TSS to ensure it describes how the keys are managed in volatile memory. This description includes details of how each identified key is introduced into volatile memory (e.g. by derivation from user input, or by unwrapping a wrapped key stored in non-volatile memory) and how they are overwritten.

The evaluator shall check to ensure the TSS lists each type of key that is stored, and identifies the memory type (volatile or non-volatile) where key material is stored.

The TSS identifies and describes the interfaces that is used to service commands to read/write memory. The evaluator examines the interface description for each different media type to ensure that the interface supports the selections made by the ST Author.

If the ST makes use of the open assignment and fills in the type of pattern that is used, the evaluator examines the TSS to ensure it describes how that pattern is obtained and used. The evaluator shall verify that the pattern does not contain any CSPs.

The evaluator shall check that the TSS identifies any configurations or circumstances that may not strictly conform to the key destruction requirement.

Guidance

There are a variety of concerns that may prevent or delay key destruction in some cases. The evaluator shall check that the guidance documentation identifies configurations or circumstances that may not strictly conform to the key destruction requirement, and that this description is consistent with the relevant parts of the TSS and any other relevant Required Supplementary Information. The evaluator shall check that the guidance documentation provides guidance on situations where key destruction may be delayed at the physical layer.

For example, when the TOE does not have full access to the physical memory, it is possible that the storage may be implementing wear-leveling and garbage collection. This may create additional copies of the key that are logically inaccessible but persist physically. In this case, it is assumed the drive supports the TRIM command and implements garbage collection to destroy these persistent copies when not actively engaged in other tasks.

Drive vendors implement garbage collection in a variety of different ways, as such there is a variable amount of time until data is truly removed from these solutions. There is a risk that data may persist for a longer amount of time if it is contained in a block with other data not ready for erasure. It is assumed the operating system and file system of the OE support TRIM, instructing the non-volatile memory to erase copies via garbage collection upon their deletion.

It is assumed that if a RAID array is being used, only set-ups that support TRIM are utilized. It is assumed if the drive is connected via PCI-Express, the operating system supports TRIM over that channel. It is assumed the drive is healthy and contains minimal corrupted data and will be end of life before a significant amount of damage to drive health occurs, it is assumed there is a risk small amounts of potentially recoverable data may remain in damaged areas of the drive.

Finally, it is assumed the keys are not stored using a method that would be inaccessible to TRIM, such as being contained in a file less than 982 bytes which would be completely contained in the master file table.

Tests

For these tests the evaluator shall utilize appropriate development environment (e.g. a Virtual Machine) and development tools (debuggers, simulators, etc.) to test that keys are erased, including all copies of the key that may have been created internally by the TOE during normal cryptographic processing with that key.

- Test FCS_CKM.6/GENHW:1: Applied to each key held as plaintext in volatile memory and subject to destruction by overwrite by the TOE (whether or not the plaintext value is subsequently encrypted for storage in volatile or non-volatile memory). In the case where the only selection made for the destruction method key was removal of power, then this test is unnecessary. The evaluator shall:
 1. Record the value of the key in the TOE subject to erasure.
 2. Cause the TOE to perform a normal cryptographic processing with the key from Step #1.
 3. Cause the TOE to erase the key.
 4. Cause the TOE to stop the execution but not exit.
 5. Cause the TOE to dump the entire memory of the TOE into a binary file.
 6. Search the content of the binary file created in Step #5 for instances of the known key value from Step #1.
 7. Break the key value from Step #1 into 3 or 4 similar sized pieces and perform a search using each piece.

Steps 1-6 ensure that the complete key does not exist anywhere in volatile memory. If a copy is found, then the test fails.

Step 7 ensures that partial key fragments do not remain in memory. If a fragment is found, there is a miniscule chance that it is not within the context of a key (e.g., some random bits that happen to match). If this is the case the test should be repeated with a different key in Step #1. If a fragment is found the test fails.

- Test FCS_CKM.6/GENHW:2: Applied to each key held in non-volatile memory and subject to destruction by overwrite by the TOE. The evaluator shall use special tools (as needed), provided by the TOE developer if necessary, to view the key storage location:
 1. Record the value of the key in the TOE subject to erasure.
 2. Cause the TOE to perform a normal cryptographic processing with the key from Step #1.
 3. Cause the TOE to erase the key.
 4. Search the non-volatile memory the key was stored in for instances of the known key value from Step #1. If a copy is found, then the test fails.
 5. Break the key value from Step #1 into 3 or 4 similar sized pieces and perform a search using each piece. If a fragment is found then the test is repeated (as described for test 1 above), and if a fragment is found in the repeated test then the test fails.
- Test FCS_CKM.6/GENHW:3: Applied to each key held as non-volatile memory and subject to destruction by overwrite by the TOE. The evaluator shall use special tools (as needed), provided by the TOE developer if necessary, to view the key storage location:
 1. Record the storage location of the key in the TOE subject to erasure.
 2. Cause the TOE to perform a normal cryptographic processing with the key from Step #1.
 3. Cause the TOE to erase the key.
 4. Search the storage location in Step #1 of non-volatile memory to ensure the appropriate pattern is used.The test succeeds if the correct pattern is used to overwrite the key in the memory location. If the pattern is not found, the test fails.

FCS_CKM.6/KEK Cryptographic Key Destruction (Key Cryptographic Erase)

FCS_CKM.6/KEK

TSS

There is no TSS for this activity.

Guidance

There is no AGD for this activity.

KMD

The evaluator shall examine the TOE's keychain in the TSS/KMD and identify each instance a key is destroyed by this method. In each instance the evaluator shall verify all keys capable of decrypting the target key are destroyed in accordance with a specified key destruction method.

Tests

There is no test for this activity.

FCS_CKM.6/SW Cryptographic Key Destruction (Software TOE, 3rd Party Storage)

FCS_CKM.6/SW

TSS

(Key Management Description may be used if necessary details describe proprietary information)

The evaluator examines the TSS to ensure it describes how the keys are managed in volatile memory. This description includes details of how each identified key is introduced into volatile memory (e.g., by derivation from user input, or by unwrapping a wrapped key stored in non-volatile memory) and how they are overwritten.

The evaluator shall check to ensure the TSS lists each type of key that is stored in non-volatile memory, and identifies how the TOE interacts with the underlying platform to manage keys (e.g., store, retrieve, destroy). The description includes details on the method of how the TOE interacts with the platform, including an identification and description of the interfaces it uses to manage keys (e.g., file system APIs, platform key store APIs).

The evaluator examines the interface description for each different media type to ensure that the interface supports the selections and description in the TSS.

The evaluator shall check that the TSS identifies any configurations or circumstances that may not strictly conform to the key destruction requirement. If the ST makes use of the open assignment and fills in the type of pattern that is used, the evaluator examines the TSS to ensure it describes how that pattern is obtained and used. The evaluator shall verify that the pattern does not contain any CSPs.

Guidance

There are a variety of concerns that may prevent or delay key destruction in some cases. The evaluator shall check that the guidance documentation identifies configurations or circumstances that may not strictly conform to the key destruction requirement, and that this description is consistent with the relevant parts of the TSS and any other relevant Required Supplementary Information. The evaluator shall check that the guidance documentation provides guidance on situations where key destruction may be delayed at the physical layer.

For example, when the TOE does not have full access to the physical memory, it is possible that the storage may be implementing wear-leveling and garbage collection. This may create additional copies of the key that are logically inaccessible but persist physically. In this case, it is assumed the drive supports the TRIM command and implements garbage collection to destroy these persistent copies when not actively engaged in other tasks.

Drive vendors implement garbage collection in a variety of different ways, as such there is a variable amount of time until data is truly removed from these solutions. There is a risk that data may persist for a longer amount of time if it is contained in a block with other data not ready for erasure. It is assumed the operating system and file system of the OE support TRIM, instructing the non-volatile memory to erase copies via garbage collection upon their deletion.

It is assumed that if a RAID array is being used, only set-ups that support TRIM are utilized. It is assumed if the drive is connected via PCI-Express, the operating system supports TRIM over that channel. It is assumed the drive is healthy and contains minimal corrupted data and will be end of life before a significant amount of damage to drive health occurs, it is assumed there is a risk small amounts of potentially recoverable data may remain in damaged areas of the drive.

Finally, it is assumed the keys are not stored using a method that would be inaccessible to TRIM, such as being contained in a file less than 982 bytes which would be completely contained in the master file table.

Tests

- Test FCS_CKM.6/SW:1: Applied to each key held as plaintext in volatile memory and subject to destruction by overwrite by the TOE (whether or not the plaintext value is subsequently encrypted for storage in volatile or non-volatile memory). In the case where the only selection made for the destruction method key was removal of power, then this test is unnecessary. The evaluator shall:
 1. Record the value of the key in the TOE subject to erasure.
 2. Cause the TOE to perform a normal cryptographic processing with the key from Step #1.
 3. Cause the TOE to erase the key.
 4. Cause the TOE to stop the execution but not exit.
 5. Cause the TOE to dump the entire memory of the TOE into a binary file.
 6. Search the content of the binary file created in Step #5 for instances of the known key value from Step #1.
 7. Break the key value from Step #1 into 3 or 4 similar sized pieces and perform a search using each piece.

Steps 1-6 ensure that the complete key does not exist anywhere in volatile memory. If a copy is found, then the test fails.

Step 7 ensures that partial key fragments do not remain in memory. If a fragment is found, there is a miniscule chance that it is not within the context of a key (e.g., some random bits that happen to match). If this is the case the test should be repeated with a different key in Step #1. If a fragment is found the test fails.

The following tests apply only to selection a), since the TOE in this instance has more visibility into what is happening within the underlying platform (e.g., a logical view of the media). In selection b), the TOE has no visibility into the inner workings and completely relies on the underlying platform, so there is no reason to test the TOE beyond test 1.

For selection a), the following tests are used to determine the TOE is able to request the platform to overwrite the key with a TOE supplied pattern.

- Test FCS_CKM.6/SW:2: Applied to each key held in non-volatile memory and subject to destruction by overwrite by the TOE. The evaluator shall use a tool that provides a logical view of the media (e.g., MBR file system):
 1. Record the value of the key in the TOE subject to erasure.
 2. Cause the TOE to perform a normal cryptographic processing with the key from Step #1.
 3. Cause the TOE to erase the key.
 4. Search the logical view that the key was stored in for instances of the known key value from Step #1. If a copy is found, then the test fails.
 5. Break the key value from Step #1 into 3 or 4 similar sized pieces and perform a search using each piece. If a fragment is found then the test is repeated (as described for Use Case 1 test 1 above), and if a fragment is found in the repeated test then the test fails.
- Test FCS_CKM.6/SW:3: Applied to each key held as non-volatile memory and subject to destruction by overwrite by the TOE. The evaluator shall use a tool that provides a logical view of the media:
 1. Record the logical storage location of the key in the TOE subject to erasure.
 2. Cause the TOE to perform a normal cryptographic processing with the key from Step #1.
 3. Cause the TOE to erase the key.
 4. Read the logical storage location in Step #1 of non-volatile memory to ensure the appropriate pattern is used.

The test succeeds if the correct pattern is used to overwrite the key in the memory location. If the pattern is not found, the test fails.

FCS_CKM.6/TOEHW Cryptographic Key Destruction (TOE-Controlled Hardware)

TSS

(Key Management Description may be used if necessary details describe proprietary information)

The evaluator examines the TSS to ensure it describes how the keys are managed in volatile memory. This description includes details of how each identified key is introduced into volatile memory (e.g., by derivation from user input, or by unwrapping a wrapped key stored in non-volatile memory) and how they are overwritten.

The evaluator shall examine the TSS to ensure it describes the method that is used by the memory controller to write and read memory from each type of memory listed. The purpose here is to provide a description of how the memory controller works so one can determine exactly how keys are written to memory. The description would include how the data is written to and read from memory (e.g., block level, cell level), mechanisms for copies of the key that could potentially exist (e.g., a copy with parity bits, a copy without parity bits, any mechanisms that are used for redundancy).

The evaluator shall examine the TSS to ensure it describes the destruction procedure for each key that has been identified. If different types of memory are used to store the keys, the evaluator shall check to ensure that the TSS identifies the destruction procedure for each memory type where keys are stored (e.g., key X stored in flash memory is destroyed by overwriting once with zeros, key X' stored in EEPROM is destroyed by a overwrite consisting of a pseudo random pattern – the EEPROM used in the TOE uses a wear-leveling scheme as described).

If the ST makes use of the open assignment and fills in the type of pattern that is used, the evaluator examines the TSS to ensure it describes how that pattern is obtained and used. The evaluator shall verify that the pattern does not contain any CSPs.

The evaluator shall check that the TSS identifies any configurations or circumstances that may not strictly conform to the key destruction requirement.

Upon completion of the TSS examination, the evaluator understands how all the keys (and potential copies) are destroyed.

Guidance

There are a variety of concerns that may prevent or delay key destruction in some cases. The evaluator shall check that the guidance documentation identifies configurations or circumstances that may not strictly conform to the key destruction requirement, and that this description is consistent with the relevant parts of the TSS and any other relevant Required Supplementary Information. The evaluator shall check that the guidance documentation provides guidance on situations where key destruction may be delayed at the physical layer.

For example, when the TOE does not have full access to the physical memory, it is possible that the storage may be implementing wear-leveling and garbage collection. This may create additional copies of the key that are logically inaccessible but persist physically. In this case, it is assumed the drive supports the TRIM command and implements garbage collection to destroy these persistent copies when not actively engaged in other tasks.

Drive vendors implement garbage collection in a variety of different ways, as such there is a variable amount of time until data is truly removed from these solutions. There is a risk that data may persist for a longer amount of time if it is contained in a block with other data not ready for erasure. It is assumed the operating system and file system of the OE support TRIM, instructing the non-volatile memory to erase copies via garbage collection upon their deletion.

It is assumed that if a RAID array is being used, only set-ups that support TRIM are utilized. It is assumed if the drive is connected via PCI-Express, the operating system supports TRIM over that channel. It is assumed the drive is healthy and contains minimal corrupted data and will be end of life before a significant amount of damage to drive health occurs, it is assumed there is a risk small amounts of potentially recoverable data may remain in damaged areas of the drive.

Finally, it is assumed the keys are not stored using a method that would be inaccessible to TRIM, such as being contained in a file less than 982 bytes which would be completely contained in the master file table.

For destruction on wear-leveled memory, if a time period is required before is processed destruction the ST author shall provide an estimated range.

Tests

For these tests the evaluator shall use appropriate development environment (e.g. a Virtual Machine) and development tools (debuggers, simulators, etc.) to test that keys are erased, including all copies of the key that may have been created internally by the TOE during normal cryptographic processing with that key.

For destruction on wear-leveled memory, if a time period is required before is evaluator shall wait that amount of time after erasing the key in tests 2 and 3.

- Test FCS_CKM.6/TOEHW:1: Applied to each key held as plaintext in volatile memory and subject to destruction by overwrite by the TOE (whether or not the plaintext value is subsequently encrypted for storage in volatile or non-volatile memory). In the case where the only selection made for the destruction method key was removal of power, then this test is unnecessary. The evaluator shall:
 1. Record the value of the key in the TOE subject to erasure.
 2. Cause the TOE to perform a normal cryptographic processing with the key from Step #1.
 3. Cause the TOE to erase the key.
 4. Cause the TOE to stop the execution but not exit.
 5. Cause the TOE to dump the entire memory of the TOE into a binary file.
 6. Search the content of the binary file created in Step #5 for instances of the known key value from Step #1.
 7. Break the key value from Step #1 into 3 or 4 similar sized pieces and perform a search using each piece.

Steps 1-6 ensure that the complete key does not exist anywhere in volatile memory. If a copy is found, then the test fails.

Step 7 ensures that partial key fragments do not remain in memory. If a fragment is found, there is a miniscule chance that it is not within the context of a key (e.g., some random bits that happen to match). If this is the case the test should be repeated with a different key in Step #1. If a fragment is found the test fails.

- Test FCS_CKM.6/TOEHW:2: Applied to each key held in non-volatile memory and subject to destruction by overwrite by the TOE. The evaluator shall use special tools (as needed), provided by the TOE developer if necessary, to view the key storage location:
 1. Record the value of the key in the TOE subject to erasure.
 2. Cause the TOE to perform a normal cryptographic processing with the key from Step #1.
 3. Cause the TOE to erase the key.
 4. Search the non-volatile memory the key was stored in for instances of the known key value from Step #1. If a copy is found, then the test fails.
 5. Break the key value from Step #1 into 3 or 4 similar sized pieces and perform a search using each piece. If a fragment is found then the test is repeated (as described for Use test 1 above), and if a fragment is found in the repeated test then the test fails.
- Test FCS_CKM.6/TOEHW:3: Applied to each key held as non-volatile memory and subject to destruction by overwrite by the TOE. The evaluator shall use special tools (as needed), provided by the TOE developer if necessary, to view the key storage location:
 1. Record the logical storage location of the key in the TOE subject to erasure.
 2. Cause the TOE to perform a normal cryptographic processing with the key from Step #1.
 3. Cause the TOE to erase the key.
 4. Read the logical storage location in Step #1 of non-volatile memory to ensure the appropriate pattern is used.

The test succeeds if the correct pattern is used to overwrite the key in the memory location. If the pattern is not found, the test fails.

FCS_COP.1/KeyEncap Cryptographic Operation - Key Encapsulation

FCS_COP.1/KeyEncap

TSS

The evaluator shall ensure that the TSS documents that the selection of the key size is sufficient for the security strength of the key encapsulated.

The evaluator shall examine the TSS to verify that any one-time values such as nonces or masks are constructed and used in accordance with the relevant standards.

Guidance

There are no AGD evaluation activities for this SFR.

KMD

There are no KMD evaluation activities for this SFR.

Tests

The following tests may require the developer to provide access to a test platform that provides the evaluator with tools that are typically not found on factory products.

The following tests are conditional based upon the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

KAS1 [RSASVE single-party]

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
KAS1	RSASVE	[selection: 3072, 4096, 6144, 8192] bits	NIST SP 800-56B Revision 2 (Section 7.2.1)

To test the TOE's implementation of the of KAS1 RSASVE Single-Party Key Encapsulation, the evaluator shall perform the Algorithm Functional Test and Validation Test using the following input parameters:

- RSA Private key format [Basic with fixed public exponent, Prime Factor with fixed public exponent, Chinese Remainder Theorem with fixed public exponent, Basic with random public exponent, Prime Factor with random public exponent, Chinese Remainder Theorem with random public exponent]
- Modulo value [3072, 4096, 6144, 8192]
- Role [initiator, responder]
- Key confirmation supported [yes, no]

The evaluator shall generate a test group (i.e. set of tests) for each parameter value of the above parameter type with the largest number of supported values. For example, if the TOE supports all six key formats, then the evaluator shall generate six test groups. Each of the above supported parameter values must be included in at least one test group.

Regardless of how many parameter values are supported, there must be at least two test groups.

Half of the test groups are designated as Algorithm Functional Tests (AFT) and the remainder are designated as Validation Tests (VAT). If there is an odd number of groups, then the extra group is designated randomly as either AFT or VAT.

If there are only two test groups, in addition to the above, one shall act as an initiator, and the other as a responder, if supported.

Algorithm Functional Test

For each test group designated as AFT, the evaluator shall generate 10 test cases using random data (except for a fixed public exponent, if supported). The resulting shared secrets shall be compared with those generated by a known-good implementation using the same inputs.

Validation Test

For each test group designated as VAT, the evaluator shall generate 25 test cases are using random data (except for a fixed public exponent, if supported). Of the 25 test cases:

- Two test cases must have a shared secret with a leading nibble of 0s,
- Two test cases have modified derived key material,
- Two test cases have modified tags, if key confirmation is supported,
- Two test cases have modified MACs, if key confirmation is supported, and
- The remaining test cases are not modified.

To determine correctness, the evaluator shall confirm that the resulting 25 shared secrets correspond as expected for both the modified and unmodified values.

KTS-OAEP [RSA-OAEP]

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
KTS-OAEP	RSA-OAEP	[selection: 3072, 4096, 6144, 8192] bits	NIST SP 800-56B Revision 2 (Sections 6.3 & 9)

To test the TOE’s implementation of the of KTS-OAEP, the evaluator shall perform the Algorithm Functional Test and Validation Test using the following input parameters:

- Roles [initiator, receiver]
- Private Key format [Basic with fixed public exponent, Prime Factor with fixed public exponent, Chinese Remainder Theorem with fixed public exponent, Basic with random public exponent, Prime Factor with random public exponent, Chinese Remainder Theorem with random public exponent]
- Supported modulus values [3072, 4096, 6144, 8192]
- Key confirmation supported [yes, no]

The evaluator shall generate a test group (i.e. set of tests) for each parameter value of the above parameter type with the largest number of supported values. For example, if the TOE supports all six key formats, then the evaluator shall generate six test groups. Each of the above supported parameter values must be included in at least one test group.

Regardless of how many parameter values are supported, there must be at least two test groups.

Half of the test groups are designated as Algorithm Functional Tests (AFT) and the remainder are designated as Validation Tests (VAT). If there is an odd number of groups, then the extra group is designated randomly as either AFT or VAT.

If there are only two test groups, in addition to the above, one shall act as an initiator, and the other as a responder, if supported.

Algorithm Functional Test

For each test group designated as AFT, the evaluator shall generate 10 test cases using random data (except for a fixed public exponent, if supported). The resulting shared secrets shall be compared with those generated by a known-good implementation using the same inputs.

Validation Test

For each test group designated as VAT, the evaluator shall generate 25 test cases are using random data (except for a fixed public exponent, if supported). Of the 25 test cases:

- Two test cases must have a shared secret with a leading nibble of 0s,
- Two test cases have modified derived key material,
- Two test cases have modified tags, if key confirmation is supported,
- Two test cases have modified MACs, if key confirmation is supported, and
- The remaining test cases are not modified.

To determine correctness, the evaluator shall confirm that the resulting 25 shared secrets correspond as expected for both the modified and unmodified values.

ML-KEM Key Encapsulation

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
ML-KEM	ML-KEM	Parameter set = ML-KEM-1024	NIST FIPS PUB 203

To test the TOE’s implementation of ML-KEM key encapsulation/decapsulation, the evaluator shall perform the Encapsulation Test and the Decapsulation Test using the following input parameters:

- Encapsulation Parameters:
 - Parameter set [ML-KEM-1024]
 - Previously generated encapsulation key (*ek*)
 - Random value (*m*) [32 bytes]
- Decapsulation Parameters:
 - Parameter set [ML-KEM-1024]
 - Previously generated decapsulation key (*dk*)
 - Previously generated ciphertext (*c*) [32 bytes]

Encapsulation Test

For each supported parameter set the evaluator shall generate 25 test cases consisting of an encapsulation key *ek* and random value *m*. For each test case the valuator shall require the implementation under test to generate the corresponding shared secret *k* and ciphertext *c*. To determine correctness, the evaluator shall compare the resulting values with those generated using a known-good implementation using the same inputs.

Encapsulation Key Check (if supported)

The evaluator shall generate 10 encapsulation keys such that:

- Five of the encapsulation keys are valid, and
- Five of the encapsulation keys are modified such that a value in the noisy linear system is encoded into the key as a value greater than Q .

The evaluator shall invoke the TOE's Encapsulation Key Check functionality to determine the validity of the 10 keys. The unmodified keys should be determined valid, and the modified keys should be determined invalid.

Decapsulation Key Check (if supported)

The evaluator shall generate 10 decapsulation keys such that:

- Five of the decapsulation keys are valid, and
- Five of the decapsulation keys are modified such that the concatenated values $ek||H(ek)$ will no longer match by modifying $H(ek)$ to be a different value.

The evaluator shall invoke the TOE's Decapsulation Key Check functionality to determine the validity of the 10 keys. The unmodified keys should be determined valid, and the modified keys should be determined invalid.

Decapsulation Test

For each supported parameter set the evaluator shall use a single previously generated decapsulation key dk and generate 10 test cases consisting of valid and invalid ciphertexts c . For each test case the evaluator shall require the implementation under test to generate the corresponding shared secret k whether or not the ciphertext is valid. To determine correctness, the evaluator shall compare the resulting values with those generated using a known-good implementation using the same inputs.

FCS_COP.1/KeyedHash Cryptographic Operation (Keyed Hash Algorithm)

FCS_COP.1/KeyedHash

TSS

The evaluator shall examine the TSS to ensure that the size of the key is sufficient for the desired security strength of the output.

The evaluator shall examine the TSS to verify that if HMAC-SHA-256 is selected, that it is being used only as a PRF in a key derivation function or RBG.

Guidance

There is no AGD for this activity.

KMD

There is no KMD for this activity.

Tests

The following tests are conditional based upon the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

HMAC

Keyed Hash Algorithm	Cryptographic Key Sizes	List of Standards
HMAC-SHA-256	256 bits	[selection: ISO/IEC 9797-2:2021 (Section 7 “MAC Algorithm 2”), FIPS PUB 198-1]
HMAC-SHA-384	[selection: (ISO, FIPS) 384, (FIPS) 256] bits	[selection: ISO/IEC 9797-2:2021 (Section 7 “MAC Algorithm 2”), FIPS PUB 198-1]
HMAC-SHA-512	[selection: (ISO, FIPS) 512, (FIPS) 384, 256] bits	[selection: ISO/IEC 9797-2:2021 (Section 7 “MAC Algorithm 2”), FIPS PUB 198-1]

To test the TOE’s ability to generate keyed hashes using HMAC the evaluator shall perform the Algorithm Functional Test for each combination of claimed HMAC algorithm the following parameters:

- Hash function [SHA-256, SHA-384, SHA-512]
- Key length [8-65536] bits by 8s
- MAC length [32-[digest size of hash function (256, 384, 512)]] bits

Algorithm Functional Test

For each supported Hash function the evaluator shall generate 150 test cases using random input messages of 128 bits, random supported key lengths, random keys, and random supported MAC lengths such that across the 150 test cases:

- The key length includes the minimum, the maximum, a key length equal to the block size, and key lengths that are both larger and smaller than the block size.
- The MAC size includes the minimum, the maximum, and two other random values.

The evaluator shall compare the output against results generated by a known-good implementation with the same input.

FCS_COP.1/KeyEnc Cryptographic Operation (Key Encryption)

FCS_COP.1/KeyEnc
TSS

The evaluator shall examine the TSS to ensure that it describes the construction of any IVs, tweak values, and counters in conformance with the relevant specifications.

If a GCM mode algorithm is selected, then the evaluator shall examine the TOE summary specification to confirm that it describes how the IV is generated and that the same IV is never reused to encrypt different plaintext pairs under the same key. The evaluator shall also confirm that for each invocation of GCM, the length of the plaintext is at most $(2^{32})-2$ blocks.

If AES-XTS is claimed then the evaluator shall examine the TSS to verify that the TOE creates full-length keys by methods that ensure that the two key halves are distinct. The evaluator shall confirm the TSS describes the block of data containing the key and that is full block of data in alignment with selected the AES standard.

Guidance

If multiple key encryption modes are supported, the evaluator examines the guidance documentation to determine that the method of choosing a specific mode/key size by the end user is described.

KMD

The evaluator shall examine the vendor's KMD to verify that it includes a description of how key encryption will be used as part of the key chain.

Tests

The AES test should be followed in FCS_COP.1/SKC Cryptographic Operation - Symmetric Key Cryptography.

FCS_COP.1/KeyWrap Cryptographic Operation - Key Wrapping

FCS_COP.1/KeyWrap

TSS

The evaluator shall ensure that the TSS documents that the selection of the key size is sufficient for the security strength of the key wrapped.

The evaluator shall examine the TSS to ensure that it describes the construction of any IVs, nonces, and MACs in conformance with the relevant specifications.

If a CCM mode algorithm is selected, then the evaluator shall examine the TOE summary specification to confirm that it describes how the nonce is generated and that the same nonce is never reused to encrypt different plaintext pairs under the same key.

If a GCM mode algorithm is selected, then the evaluator shall examine the TOE summary specification to confirm that it describes how the IV is generated and that the same IV is never reused to encrypt different plaintext pairs under the same key. The evaluator shall also confirm that for each invocation of GCM, the length of the plaintext is at most $(2^{32})-2$ blocks.

Guidance

There is no AGD for this activity.

KMD

The evaluator shall review the KMD to ensure that all keys are wrapped using the approved method and a description of when the key wrapping occurs.

Tests

For tests of AES-GCM, see testing for FCS_COP.1/SKC.

The following tests are conditional based upon the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

AES-KW

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
AES-KW	AES in KW mode	256 bits	[selection: ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES] [selection: ISO/IEC 19772:2020 (clause 6), NIST SP 800-38F (Section 6.2)] [KW mode]

To test the TOE's ability to wrap keys using AES in Key Wrap mode the evaluator shall perform the Algorithm Functional Tests using the following input parameters:

- Key size [256] bits

- Keyword cipher type [cipher, inverse]
- Payload sizes [128-4096] bits by 64s

Algorithm Functional Test

The evaluator shall generate 100 encryption test cases using random data for each combination of claimed key size, keyword cipher type, and six supported payload sizes such that the payload sizes include the minimum, the maximum, two that are divisible by 128, and two that are not divisible by 128.

The results shall be compared with those generated by a known-good implementation using the same inputs.

The evaluator shall generate 100 decryption test cases using the same parameters as above, but with 20 of each 100 test cases having modified ciphertext to produce an incorrect result. To determine correctness, the evaluator shall confirm that the results correspond as expected for both the modified and unmodified values.

AES-KWP

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
AES-KWP	AES in KWP mode	256 bits	[selection: ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES] NIST SP 800-38F (Section 6.3) [KWP mode]

To test the TOE's ability to wrap keys using AES in Key Wrap with Padding mode with padding the evaluator shall perform the Algorithm Functional Tests using the following input parameters:

- Key size [256] bits
- Keyword cipher type [cipher, inverse]
- Payload sizes [8-4096] bits by 8s

Algorithm Functional Test

The evaluator shall generate 100 encryption test cases using random data for each combination of claimed key size, keyword cipher type, and six supported payload sizes such that the payload sizes include the minimum, the maximum, two that are divisible by 128, and two that are not divisible by 128.

The results shall be compared with those generated by a known-good implementation using the same inputs.

The evaluator shall generate 100 decryption test cases using the same parameters as above, but with 20 of each 100 test cases having modified ciphertext to produce an incorrect result. To determine correctness, the evaluator shall confirm that the results correspond as expected for both the modified and unmodified values.

AES-CCM

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards

AES-CCM	AES in CCM mode with non-repeating nonce, minimum size of 64 bits	256 bits	[selection: ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES] [selection: ISO/IEC 19772:2020 (Clause 7), NIST SP 800-38C] [CCM]
---------	---	----------	---

To test the TOE's implementation of AES-CCM authenticated encryption functionality the evaluator shall perform the Algorithm Functional Tests described below using the following input parameters:

- Key Size [256] bits
- Associated data size [0-65536] bits in increments of 8
- Payload size [0-256] bits in increments of 8
- IV/Nonce size [64-104] bits in increments of 8
- Tag size [32-128] bits in increments of 16

Algorithm Functional Tests

Unless otherwise specified, the following tests should use random data, a tag size of 128 bits, IV/Nonce size of 104 bits, payload size of 256 bits, and associated data size of 256 bits. If any of these values are not supported, any supported value may be used. The evaluator shall compare the output from each test case against results generated by a known-good implementation with the same input parameters.

Variable Associated Data Test

For each claimed key size, and for each supported associated data size from 0 through 256 bits in increments of 8 bits, the TOE must be tested by encrypting 10 test cases using all random data. In addition, for each key size, the TOE must be tested by encrypting 10 cases with associated data lengths of 65536 bits, if supported.

Variable Payload Test

For each claimed key size, and for each supported payload size from 0 through 256 bits in increments of 8 bits, the TOE must be tested by encrypting 10 test cases using all random data.

Variable Nonce Test

For each claimed key size, and for each supported IV/Nonce size from 64 through 104 bits in increments of 8 bits, the TOE must be tested by encrypting 10 test cases using all random data.

Variable Tag Test

For each claimed key size, and for each supported tag size from 32 through 128 bits in increments of 16 bits, the TOE must be tested by encrypting 10 test cases using all random data.

Decryption Verification Test

For each claimed key size, for each supported associated data size from 0 through 256 bits in increments of 8 bits, for each supported payload size from 0 through 256 bits in increments of 8 bits, for each supported IV/Nonce size

from 64 through 104 bits in increments of 8 bits, and for each supported tag size from 32 through 128 bits in increments of 16 bits, the TOE must be tested by decrypting 10 test cases using all random data.

FCS_CKM.5 Cryptographic Key Derivation

FCS_CKM.5

TSS

The evaluator shall verify the TSS includes a description of the key derivation function and shall verify the key derivation function uses an approved derivation mode and key expansion algorithm according to list of standards.

Guidance

The evaluator shall verify that the Guidance instructs the administrator how to set any configurable parameters, such as context strings, salts, and IVs.

The evaluator shall verify that the Guidance instructs the administrator how to configure the TOE to choose specific PRFs, modes, and parameters.

KMD

The evaluator shall verify that the KMD describes and documents:

- that the security strengths of the Pseudo-Random functions for the key derivation methods are sufficient for the security strength of the keys derived through those methods.
- that the security strengths of the input parameters are sufficient for the security strength of the keys derived through these methods.
- that, if concatenated keys or intermediary keys are input parameters, the KMD describes the sources of the keys, and the order in which they are concatenated, along with any other values that are concatenated with them. This may occur in instances when input keying material for the KDF comes from two independent sources, for example, a client and a server.
- that, for KDF-MAC-1S, KDF-MAC-2S, and KDF-KMAC, the KMD documents that for each invocation of a KDF that reuses the same input shared secret or key, each invocation must use a distinct context string, IV, or salt. The KMD must also describe the composition and sizes of these input parameters. The evaluator must ensure that the context string, IV, and salt are generated in conformance with the relevant standards.
- that, if the TOE uses the derived key in a key chain/hierarchy, that KMD describes how the key is used as part of the key chain/hierarchy.

Tests

The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

KDF in Counter Mode

Key Type	Input Parameters	Key Derivation Algorithm	Key Sizes	List of Standards
KDF-CTR	[selection: Direct Generation from a Random Bit Generator as specified in FCS_RBG.1, Concatenated keys]	KPF2 - KDF in Counter Mode using [selection: AES-256-CMAC, HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512] as the PRF	[selection: 256, 384, 512] bits	[selection: ISO/IEC 11770-6:2016 (Subclause 7.3.2) [KPF2], NIST SP 800-108 Revision 1 Update 1 (Section 4.1) [KDF in Counter Mode]]

To test the TOE's ability to derive cryptographic keys using KDF in Counter Mode/KDF2, the evaluator shall perform the Counter KDF Algorithm Functional Test using the following input parameters:

- Pseudo Random Function (PRF) [AES-256-CMAC, HMAC-SHA-256, HMAC-SHA-512]
- Derived key length [256, 512] bits
- Location of the counter [after fixed data, before fixed data, middle fixed data]
- Counter length [8, 16, 24, 32] bits

Counter KDF Algorithm Functional Test

For each supported combination of the above input parameters the evaluator shall require the implementation under test to derive two keys using random data. The evaluator shall compare the resulting keys with keys generated using a known-good implementation using the same input parameters.

KDF in Feedback Mode

Key Type	Input Parameters	Key Derivation Algorithm	Key Sizes	List of Standards
KDF-FB	[selection: Direct Generation from a Random Bit Generator as specified in FCS_RBG.1, Concatenated keys]	KPF3 - KDF in Feedback Mode using [selection: AES-256-CMAC, HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512] as the PRF	[selection: 256, 384, 512] bits	[selection: ISO/IEC 11770-6:2016 (Subclause 7.3.3) [KPF3], NIST SP 800-108 Revision 1 Update 1 (Section 4.2) [KDF in Feedback Mode]]

To test the TOE's ability to derive cryptographic keys using KDF in Feedback Mode/KDF3, the evaluator shall perform the Feedback KDF Algorithm Functional Test using the following input parameters:

- Pseudo Random Function (PRF) [AES-256-CMAC, HMAC-SHA-256, HMAC-SHA-512]
- Derived key length [256, 512] bits
- Location of the counter [none, after fixed data, before fixed data, before iterator]
- Counter length [0, 8, 16, 24, 32] bits

Feedback KDF Algorithm Functional Test

For each supported combination of the above input parameters the evaluator shall require the implementation under test to derive two keys using random data. The evaluator shall compare the resulting keys with keys generated using a known-good implementation using the same input parameters.

KDF in Double-Pipeline Iteration Mode

Key Type	Input Parameters	Key Derivation Algorithm	Key Sizes	List of Standards
KDF-DPI	[selection: Direct Generation from a Random Bit Generator as specified in FCS_RBG.1, Concatenated keys]	KPF4 - KDF in Double-Pipeline Iteration Mode using [selection: HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512] as the PRF	[selection: 256, 384, 512] bits	[selection: ISO/IEC 11770-6:2016 (Subclause 7.3.4) [KPF4], NIST SP 800-108 Revision 1 Update 1 (Section 4.3) [KDF in Double-Pipeline Iteration Mode]]

To test the TOE's ability to derive cryptographic keys using KDF in Double Pipeline Iteration Mode/KDF4, the evaluator shall perform the Double Pipeline Iteration KDF Algorithm Functional Test using the following input parameters:

- Pseudo Random Function (PRF) [HMAC-SHA-256, HMAC-SHA-512]
- Derived key length [256, 512] bits
- Location of the counter [none, after fixed data, before fixed data, before iterator]
- Counter length [0, 8, 16, 24, 32] bits

Double Pipeline Iteration KDF Algorithm Functional Test

For each supported combination of the above input parameters the evaluator shall require the implementation under test to derive two keys using random data. The evaluator shall compare the resulting keys with keys generated using a known-good implementation using the same input parameters.

KDF by Hashing a Shared Secret

Key Type	Input Parameters	Key Derivation Algorithm	Key Sizes	List of Standards
KDF-HASH	Shared secret	Hash function [selection: SHA-384, SHA-512]	[selection: 256, 384, 512] bits	NIST SP 800-56C Revision 2 (Section 4.1, Option 1) [One-Step Key Derivation]

To test the TOE's ability to derive cryptographic keys by hashing a shared secret (a.k.a. One-Step HASH-based Key Derivation), the evaluator shall perform the Algorithm Functional Test using the following input parameters:

- Auxiliary Function [SHA-384, SHA-512]
- Derived key length [256, 384, 512] bits

Algorithm Functional Test

For each supported fixed information pattern and combination of the above input parameters the evaluator shall require the implementation under test to derive 15 keys using random data for a shared secret that is the same size as the derived key. The evaluator shall compare the resulting keys with keys derived using a known-good implementation using the same fixed information patterns and input parameters.

One-Step MAC-based KDF

Key Type	Input Parameters	Key Derivation Algorithm	Key Sizes	List of Standards
KDF-MAC-1S	Shared secret, salt, output length, fixed information	Keyed Hash function [selection: HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512]	[selection: 256, 384, 512] bits	NIST SP 800-56C Revision 2 (Section 4.1, Options 2, 3) [One-Step Key Derivation]

To test the TOE's ability to derive cryptographic keys using One-Step MAC-based Key Derivation, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

- Auxiliary Function [HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512]

- Salt [0s, random]
- Derived key length [256, 384, 512] bits
- Fixed information pattern

Algorithm Functional Test

For each supported fixed information pattern and combination of the above input parameters the evaluator shall require the implementation under test to derive 15 keys using random data for a shared secret. The evaluator shall compare the resulting keys with keys derived using a known-good implementation using the same fixed information patterns and input parameters.

Two-Step MAC-based KDF

Key Type	Input Parameters	Key Derivation Algorithm	Key Sizes	List of Standards
KDF-MAC-2S	Shared secret, salt, IV, output length, fixed information, and [selection: auxiliary shared secret, no other parameters]	MAC Step [selection: HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512] as randomness extraction and; KDF Step [selection: KDF-CTR, KDF-FB, KDF-DPI]	[selection: 256, 384, 512] bits	NIST SP 800-56C Revision 2 (Section 5) [Two-Step Key Derivation]

To test the TOE's ability to derive cryptographic keys using Two-Step MAC-based Key Derivation, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

- MAC mode [HMAC-SHA-256; HMAC-SHA-384, HMAC-SHA-512]
- KDF Mode [Counter, feedback, Double Pipeline Iteration]
- Salt [0s, random]
- Length of shared secret [224-65535]
- Length of Auxiliary Shared Secret [0, 112-65535]
- Derived key length [256, 384, 512] bits
- Fixed information pattern
- Counter location [none, before fixed data, after fixed data, before iterator]
- Counter length [0, 8, 16, 24, 32]

Algorithm Functional Test

The evaluator shall define a test group for each supported combination of KDF mode, MAC mode, fixed information pattern, derived key length, counter location, counter length, salt method, and five random pairs of shared secrets & auxiliary secrets (if supported) such that collectively the minimum length, maximum length and three random lengths of each are included in each test group. For each test group, the evaluator shall require the implementation under test to derive 25 keys using random data for a shared secret, either a random salt or a salt of all 0s, and, if supported, an auxiliary shared secret consisting of random data. The evaluator shall compare the resulting keys with keys derived using a known-good implementation using the same input parameters.

FCS_COP.1/CMAC Cryptographic Operation - CMAC

FCS_COP.1/CMAC

TSS

The evaluator shall examine the TSS to verify that the IV consists of all zeros in accordance with the relevant standards.

Guidance

There is no AGD for this activity.

Tests

The following tests may require the developer to provide access to a test platform that provides the evaluator with tools that are typically not found on factory products.

The following tests are conditional based upon the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

AES-CMAC

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
AES-CMAC	AES using CMAC mode	256 bits	[selection: ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES] [selection: ISO/IEC 9797-1:2011 (Subclause 7.6), NIST SP 800-38B] [CMAC]

To test the TOE's ability to generate MAC values using AES in CMAC mode the evaluator shall perform the CMAC Generation Test and CMAC Verification Test using the following input parameters:

- Key Size [256] bits
- Message size [0-524288] bits in increments of 8
- MAC sizes [1-128] bits

CMAC Generation Test

The evaluator shall generate eight test cases using random keys and data for each combination of the above parameters as follows:

- For each claimed key size,
- For four message sizes as follows:
 - The smallest supported message size,
 - The largest supported message size,
 - Two sizes that are divisible by the block size, and
 - Two sizes that are not divisible by the block size
- For three MAC sizes as follows:
 - The smallest supported MAC size,
 - The largest supported MAC size, and
 - Some other supported MAC size

The evaluator shall compare the output from each test case against results generated by a known- good implementation with the same input parameters.

CMAC Verification Test

The evaluator shall generate 20 test cases using random keys and data for each combination of the above parameters as follows:

- For each claimed key size,
- For four message sizes as follows:
 - The smallest supported message size,
 - The largest supported message size,
 - Two sizes that are divisible by the block size, and
 - Two sizes that are not divisible by the block size
- For three MAC sizes as follows:
 - The smallest supported MAC size,
 - The largest supported MAC size, and
 - Some other supported MAC size

The evaluator shall modify the tag such that 25% of the test cases in each group of 20 test cases should fail.

The evaluator shall determine that the verification fails for the test cases with modified inputs, and succeeds for those with unmodified inputs.

FCS_RBG.1 Cryptographic Operation (Random Bit Generation)

FCS_RBG.1.1

TSS

The evaluator shall verify that the TSS identifies the DRBGs used by the TOE.

Guidance

If the DRBG functionality is configurable, the evaluator shall verify that the operational guidance includes instructions on how to configure this behavior.

Tests

The evaluator shall perform the following tests:

The evaluator shall perform 15 trials for the DRBG implementation. If the DRBG is configurable, the evaluator shall perform 15 trials for each configuration. The evaluator shall also confirm that the operational guidance contains appropriate instructions for configuring the DRBG functionality.

If the DRBG has prediction resistance enabled, each trial consists of (1) instantiate DRBG, (2) generate the first block of random bits (3) generate a second block of random bits (4) uninstantiate. The evaluator verifies that the second block of random bits is the expected value. The evaluator shall generate eight input values for each trial. The first is a count (0 – 14). The next three are entropy input, nonce, and personalization string for the instantiate operation. The next two are additional input and entropy input for the first call to generate. The final two are additional input and entropy input for the second call to generate. These values are randomly generated. "generate one block of random bits" means to generate random bits with number of returned bits equal to the Output Block Length (as defined in NIST SP 800-90A).

If the DRBG does not have prediction resistance, each trial consists of (1) instantiate DRBG, (2) generate the first block of random bits (3) reseed, (4) generate a second block of random bits (5) uninstantiate. The evaluator verifies that the second block of random bits is the expected value. The evaluator shall generate eight input values for each trial. The first is a count (0 – 14). The next three are entropy input, nonce, and personalization string for the instantiate operation. The fifth value is additional input to the first call to generate. The sixth and seventh are additional input and entropy input to the call to reseed. The final value is additional input to the second generate call.

The following list contains more information on some of the input values to be generated/selected by the evaluator.

- **Entropy input:** The length of the entropy input value must equal the seed length.
- **Nonce:** If a nonce is supported (CTR_DRBG with no Derivation Function does not use a nonce), the nonce bit length is one-half the seed length.

- **Personalization string:** The length of the personalization string must be less than or equal to seed length. If the implementation only supports one personalization string length, then the same length can be used for both values. If more than one string length is support, the evaluator shall use personalization strings of two different lengths. If the implementation does not use a personalization string, no value needs to be supplied.
- **Additional input:** The additional input bit lengths have the same defaults and restrictions as the personalization string lengths.

FCS_RBG.1.2

Documentation will be produced - and the evaluator shall perform the activities - in accordance with Appendix D, "Entropy Documentation and Assessment" and the , Clarification to the Entropy Documentation and Assessment Annex.

TSS

There are no additional TSS required for this activity.

Guidance

There are no additional AGD required for this activity.

KMD

There are no additional KMD required for this activity.

Tests

There are no additional tests required for this activity.

FCS_RBG.1.3

TSS

The evaluator shall verify that the TSS identifies how the DRBG state is updated, and the situations under which this may occur.

Guidance

If the ST claims that the DRBG state can be updated on demand, the evaluator shall verify that the operational guidance has instructions for how to perform this operation.

KMD

There are no additional KMD required for this activity.

Tests

There are no additional tests required for this activity.

FCS_RBG.2 Random Bit Generation (External Seeding)

FCS_RBG.2

The evaluator shall examine the entropy documentation required by FCS_RBG.1.2 to verify that it identifies, for each DRBG function implemented by the TOE, the TSF external interface used to seed the TOE's DRBG. The evaluator shall verify that this includes the amount of sampled data and the min-entropy rate of the sampled data such that it can be determined that sufficient entropy can be made available for the highest strength keys that the TSF can generate (e.g., 256 bits). If the seed data cannot be assumed to have full entropy (e.g., the min-entropy of the sampled bits is less than 1), the evaluator shall ensure that the entropy documentation describes the method by which the TOE estimates the amount of entropy that has been accumulated to ensure that sufficient data is collected and any conditioning that the TSF applies to the output data to create a seed of sufficient size with full entropy.

TSS

There are no additional TSS required for this activity.

Guidance

There are no additional AGD required for this activity.

KMD

There are no additional KMD required for this activity.

Tests

There are no additional tests required for this activity.

FCS_RBG.3 Random Bit Generation (Internal Seeding - Single Source)

FCS_RBG.3

The evaluator shall examine the entropy documentation required by FCS_RBG.1.2 to verify that it identifies, for each DRBG function implemented by the TOE, the TSF noise source used to seed the TOE's DRBG. The evaluator shall verify that this includes the amount of sampled data and the min-entropy rate of the sampled data such that it can be determined that sufficient entropy can be made available for the highest strength keys that the TSF can generate (e.g., 256 bits). If the seed data cannot be assumed to have full entropy (e.g., the min-entropy of the sampled bits is less than 1), the evaluator shall ensure that the entropy documentation describes the method by which the TOE estimates the amount of entropy that has been accumulated to ensure that sufficient data is collected and any conditioning that the TSF applies to the output data to create a seed of sufficient size with full entropy.

TSS

There are no additional TSS required for this activity.

Guidance

There are no additional AGD required for this activity.

KMD

There are no additional KMD required for this activity.

Tests

There are no additional tests required for this activity.

FCS_RBG.4 Random Bit Generation (Internal Seeding - Multiple Sources)

FCS_RBG.4

The evaluator shall examine the entropy documentation required by FCS_RBG.1.2 to verify that it identifies, for each DRBG function implemented by the TOE, each TSF noise source used to seed the TOE's DRBG. The evaluator shall verify that this includes the amount of sampled data and the min-entropy rate of the sampled data from each data source.

TSS

There are no additional TSS required for this activity.

Guidance

There are no additional AGD required for this activity.

KMD

There are no additional KMD required for this activity.

Tests

There are no additional tests required for this activity.

FCS_RBG.5 Random Bit Generation (Combining Noise Sources)

FCS_RBG.5

Using the entropy sources specified in FCS_RBG.4, the evaluator shall examine the entropy documentation required by FCS_RBG.1.2 to verify that it describes the method by which the various entropy sources are combined into a single seed. This should include an estimation of the rate at which each noise source outputs data and whether this is dependent on any system-specific factors so that each source's relative contribution to the overall entropy is understood. The evaluator shall verify that the resulting combination of sampled data and the min-entropy rate of the sampled data is described in sufficient detail to determine that sufficient entropy can be made available for the highest strength keys that the TSF can generate (e.g., 256 bits). If the seed data cannot be assumed to have full entropy (e.g., the min-entropy of the sampled bits is less than 1), the evaluator shall ensure that the entropy documentation describes the method by which the TOE estimates the amount of entropy that has been accumulated to ensure that sufficient data is collected and any conditioning that the TSF applies to the output data to create a seed of sufficient size with full entropy.

TSS

There are no additional TSS required for this activity.

Guidance

There are no additional AGD required for this activity.

KMD

There are no additional KMD required for this activity.

Tests

There are no additional tests required for this activity.

FCS_SMC_EXT.1 Submask Combining

FCS_SMC_EXT.1

TSS

If the submasks produced from the authorization factors are XORed together to form the BEV or intermediate key, the TSS section shall identify how this is performed (e.g., if there are ordering requirements, checks performed, etc.). The evaluator shall also confirm that the TSS describes how the length of the output produced is at least the same as that of the BEV.

Guidance

There are no AGD evaluation activities for this SFR.

KMD

The evaluator shall review the KMD to ensure that an approved combination is used and does not result in the weakening or exposure of key material.

Tests

The evaluator shall perform the following test:

- Test FCS_SMC_EXT.1:1: (conditional): If there is more than one authorization factor, ensure that failure to supply a required authorization factor does not result in access to the encrypted data.

2.3.2 Protection of the TSF (FPT)

FPT_FLS.1 Failure with Preservation of Secure State

FPT_FLS.1

TSS

The evaluator shall verify that the TSF describes how the TOE enters an error state in the event of a DRBG self-test failure.

Guidance

The evaluator shall verify that the guidance documentation describes the error state that results from a DRBG self-test failure and the actions that a user or administrator should take in response to attempt to resolve the error state.

Tests

There is no Test for this activity.

FPT_FUA_EXT.1 Firmware Update Authentication

FPT_FUA_EXT.1

TSS

The evaluator shall examine the TSS to ensure that it describes how the TOE uses the RTU, what type of key or hash value, and where the value is stored on the RTU. The evaluator shall also verify that the TSS contains a description (storage location) of where the original firmware exists.

Guidance

There is no AGD for this activity.

KMD

There is no KMD for this activity.

Tests

There is no test for this activity

2.4 Evaluation Activities for Objective SFRs

The PP-Module does not define any objective requirements.

2.5 Evaluation Activities for Implementation-dependent SFRs

2.5.1 Cryptographic Support (FCS)

FCS_CKM.1/SKG Cryptographic Key Generation (Symmetric Keys)

FCS_CKM.1/SKG

TSS

The evaluator shall review the TSS to determine that a symmetric key is supported by the product, that the TSS includes a description of the protection provided by the product for this key. The evaluator shall ensure that the TSS identifies the key sizes supported by the TOE.

Guidance

The evaluator shall verify that the AGD guidance instructs the administrator how to configure the TOE to use the selected key sizes for all uses specified by the AGD documentation and defined in this cPP.

KMD

If the TOE uses a symmetric key as part of the key chain, the KMD should detail how the symmetric key is used as part of the key chain.

Tests

There are no test evaluation activities for this SFR.

3 Evaluation Activities for SARs

3.1 ADV: Development

ADV_FSP.1 Basic Functional Specification (ADV_FSP.1)

ADV_FSP.1

There are no specific evaluation activities associated with these SARs, except ensuring the information is provided. The functional specification documentation is provided to support the evaluation activities described in Section 5.1 Security Functional Requirements, and other activities described for AGD, ATE, and AVA SARs. The requirements on the content of the functional specification information is implicitly assessed by virtue of the other evaluation activities being performed; if the evaluator is unable to perform an activity because there is insufficient interface information, then an adequate functional specification has not been provided.

3.2 AGD: Guidance Documentation

AGD_OPE.1 Operational User Guidance (AGD_OPE.1)

AGD_OPE.1

Some of the contents of the operational guidance will be verified by the evaluation activities in Section 5.1 Security Functional Requirements and evaluation of the TOE according to the [CEM]. The following additional information is also required.

If cryptographic functions are provided by the TOE, the operational guidance shall contain instructions for configuring the cryptographic engine associated with the evaluated configuration of the TOE. It shall provide a warning to the administrator that use of other cryptographic engines was not evaluated nor tested during the CC evaluation of the TOE.

The documentation must describe the process for verifying updates to the TOE by verifying a digital signature – this may be done by the TOE or the underlying platform.

The evaluator shall verify that this process includes the following steps:

- Instructions for obtaining the update itself. This should include instructions for making the update accessible to the TOE (e.g., placement in a specific directory).
- Instructions for initiating the update process, as well as discerning whether the process was successful or unsuccessful. This includes generation of the digital signature. The TOE will likely contain security functionality that does not fall in the scope of evaluation under this cPP. The operational guidance shall make it clear to an administrator which security functionality is covered by the evaluation activities.

AGD_PRE.1 Preparative Procedures (AGD_PRE.1)

AGD_PRE.1

As indicated in the introduction above, there are significant expectations with respect to the documentation—especially when configuring the operational environment to support TOE functional requirements. The evaluator shall check to ensure that the guidance provided for the TOE adequately addresses all platforms claimed for the TOE in the ST.

3.3 Class ALC: Life-cycle Support

ALC_CMC.1 Labelling of the TOE (ALC_CMC.1)

ALC_CMC.1

The evaluator shall check the ST to ensure that it contains an identifier (such as a product name/version number) that specifically identifies the version that meets the requirements of the ST. Further, the evaluator shall check the operational guidance and TOE samples received for testing to ensure that the version number is consistent with that in the ST. If the vendor maintains a website advertising the TOE, the evaluator shall examine the information on the website to ensure that the information in the ST is sufficient to distinguish the product.

ALC_CMS.1 TOE CMS Coverage (ALC_CMS.1)

ALC_CMS.1

The "evaluation evidence required by the SARs" in this cPP is limited to the information in the ST coupled with the guidance provided to administrators and users under the AGD requirements. By ensuring that the TOE is specifically identified and that this identification is consistent in the ST and in the AGD guidance (as done in the evaluation activity for ALC_CMC.1), the evaluator implicitly confirms the information required by this component. Life-cycle support is targeted aspects of the developer's life-cycle and instructions to providers of applications for the developer's devices, rather than an in-depth examination of the TSF manufacturer's development and configuration management process. This is not meant to diminish the critical role that a developer's practices play in contributing to the overall trustworthiness of a product; rather, it's a reflection on the information to be made available for evaluation.

ALC_FLR.1 Basic Flaw Remediation (ALC_FLR.1)

ALC_FLR.1

The evaluator shall inspect the TSS and verify it identifies how to access the flaw remediation procedures.

ALC_FLR.2 Flaw Reporting Procedures (ALC_FLR.2)

ALC_FLR.2

The evaluator shall inspect the TSS and verify it identifies how to access the flaw remediation procedures.

The evaluator shall inspect the guidance document and verify it describes how to access the flaw remediation guidance.

ALC_FLR.3 Systematic Flaw Remediation (ALC_FLR.3)

ALC_FLR.3

The evaluator shall inspect the TSS and verify it identifies how to access the flaw remediation procedures.

The evaluator shall inspect the guidance document and verify it describes how to access the flaw remediation guidance.

3.4 Class ATE: Tests

ATE_IND.1 Independent Testing – Conformance (ATE_IND.1)

ATE_IND.1

The evaluator shall prepare a test plan and report documenting the testing aspects of the system, including any application crashes during testing. The evaluator shall determine the root cause of any application crashes and include that information in the report. The test plan covers all of the testing actions contained in the [CEM] and the body of this cPP's evaluation activities.

While it is not necessary to have one test case per test listed in an evaluation activity, the evaluator must document in the test plan that each applicable testing requirement in the ST is covered. The test plan identifies the platforms to be tested, and for those platforms not included in the test plan but included in the ST, the test plan provides a justification for not testing the platforms. This justification must address the differences between the tested platforms and the untested platforms, and make an argument that the differences do not affect the testing to be performed. It is not sufficient to merely assert that the differences have no effect; rationale must be provided. If all platforms claimed in the ST are tested, then no rationale is necessary. The test plan describes the composition of each platform to be tested, and any setup that is necessary beyond what is contained in the AGD documentation. It should be noted that the evaluator is expected to follow the AGD documentation for installation and setup of each platform either as part of a test or as a standard pre-test condition. This may include special test drivers or tools. For each driver or tool, an argument (not just an assertion) should be provided that the driver or tool will not adversely affect the performance of the functionality by the TOE and its platform.

This also includes the configuration of the cryptographic engine to be used. The cryptographic algorithms implemented by this engine are those specified by this cPP and used by the cryptographic protocols being evaluated (e.g., SSH). The test plan identifies high-level test objectives as well as the test procedures to be followed to achieve those objectives. These procedures include expected results.

The test report (which could just be an annotated version of the test plan) details the activities that took place when the test procedures were executed, and includes the actual results of the tests. This shall be a cumulative account, so

if there was a test run that resulted in a failure; a fix installed; and then a successful re-run of the test, the report would show a “fail” and “pass” result (and the supporting details), and not just the “pass” result.

3.5 Class AVA: Vulnerability Assessment

AVA_VAN.1 Vulnerability Survey (AVA_VAN.1)

AVA_VAN.1

The evaluator shall generate a report to document their findings with respect to this requirement. This report could physically be part of the overall test report mentioned in ATE_IND, or a separate document. The evaluator performs a search of public information to find vulnerabilities that have been found in similar applications with a particular focus on network protocols the application uses and document formats it parses.

The evaluator documents the sources consulted and the vulnerabilities found in the report.

For each vulnerability found, the evaluator either provides a rationale with respect to its non-applicability, or the evaluator formulates a test (using the guidelines provided in ATE_IND) to confirm the vulnerability, if suitable. Suitability is determined by assessing the attack vector needed to take advantage of the vulnerability. If exploiting the vulnerability requires expert skills and an electron microscope, for instance, then a test would not be suitable and an appropriate justification would be formulated.

4 Required Supplementary Information

This Supporting Document has no required supplementary information beyond the ST, operational guidance, and testing.

Appendix A - References

Identifier	Title
[CC]	Common Criteria for Information Technology Security Evaluation -
	• Part 1: Introduction and General Model , CCMB-2017-04-001, Version 3.1 Revision 5, April 2017.
	• Part 2: Security Functional Components , CCMB-2017-04-002, Version 3.1 Revision 5, April 2017.
	• Part 3: Security Assurance Components , CCMB-2017-04-003, Version 3.1 Revision 5, April 2017.
[FDE–EE]	collaborative Protection Profile for Full Drive Encryption – Encryption Engine, Version 3.0, March, 2026